Golden Bike Shop Integrated System

Final Design Package

Golden Bike Shop Owner: Gregg Bishop

Development Team: Kickstarters

Planned Duration: 8 weeks

Estimated Budget: $65,000

GOLDEN BIKE SHOP DESIGN PACKAGE

Table of Contents

INTRODUCTION 4

01 / CHARTER & PROCESS 5

- Overview and Context 5
- Stakeholders 5
- Scope, Constraints, Assumptions, and Risks 6
- Constraints 7
- High-Level Risks 7
- Planned Schedule 8
- Process Selection and Justification 8
- Alternatives Considered 8

02 / REQUIREMENTS 10

- System Scope 10
- Functional Requirements 11
- Non-Functional Requirements 16
- Project Charter Changes 18

03 / USER JOURNEYS & INTERFACE 19

- User Interface Designs 19
- Journey 1 19
- Journey 2 24
- Journey 3 28

04 / APPLICATION ARCHITECTURE 34

- Architectural Approach 34
- Object Model 34
- Services 38
- Data Design 40
- How the Architecture Meets the Quality Attributes 42

Case Against the Architecture 44
05 / DATA DESIGN 46
How Data Is Stored 46
Entity Relation Diagram 47
Table Descriptions 48
How Data Moves 51
Changes Made to the Week 5 Design 53
Checking Generated Model 53
06 / QUALITY & TEST PLAN 60
Introduction 60
Quality Process 60
Testing Inside the Process 61
Escaped Defects 63
Severity classifications are defined as follows: 63
Detailed Tests 64
Requirements Traceability Overview: 64
Test 1: Product Search and Inventory Availability (REQ-001) 65
Test 2: Rental Bike Availability Restrictions (REQ-004) 65
Test 3: Service Appointment Scheduling Conflicts (REQ-006) 66
Test 4: Inventory Synchronization Time (NFR-001) 67
Test 5: Role-Based Access Restrictions (NFR-003) 68
Testing limitations 69
What The Plan Misses 71
Disaster Recovery Testing 73
07 / DEPLOYMENT AND MAINTENANCE OUTLOOK 75
The same build moves through three environments without being rebuilt: 75
What Will Be Hardest to Maintain 76
08 / Discrepancy and Resolution Notice 78

INTRODUCTION

The Golden Bike Shop Integrated System is a web application that gives Golden Bike Shop one shared, real-time view of its three businesses: retail sales, bicycle rentals, and bicycle service. Today, each of the shop's four stores relies on separate systems that don't share information. Staff can't easily tell whether a product is in stock at another location, whether a rental bike is truly available, or how repairs affect rental availability and workshop capacity. Our system connects to the shop's existing point-of-sale, payment, telemetry, and workshop systems instead of replacing them.

The system is built for the people who run the shop every day:

- the owner, Gregg Bishop, who needs business value at a controlled cost
- store managers like Marcus Glover, who need a simple, easy-to-learn interface
- retail, rental, and service employees, who need accurate, up-to-date information to serve customers

Customers benefit too, through accurate stock and rental availability and the ability to check on their own bike service.

To sum it up, our team would build retail inventory lookup first. That means letting staff search for any product by UPC or name and see how many are on hand at all four stores (REQ-001), along with the store manager's detailed inventory view (REQ-002).

We chose this starting point for three reasons:

- **Revenue:** retail sales are 60% of the shop's gross revenue, and in-store traffic accounts for about 70% of sales, so this feature delivers the most value soonest.
- **Foundation:** it builds the Store, Product, and Inventory Record tables and the point-of-sale connection that online ordering, sales history, and reorder decisions all depend on.
- **Early proof:** it tests our hardest commitments first. Inventory must update within two seconds, and only managers can reach manager-level views. If either has a problem, we find out in the first iteration, not the last.

With retail work, we would move on to rentals and service bay scheduling, which carry 35% of revenue and the shop's biggest operational risk.

01 / CHARTER & PROCESS

The business case, stakeholders, constraints and evolutionary lifecycle with Agile practices establish the scope that the remaining design must honor.

Overview and Context

Golden Bike Shop is a regional business operating four physical stores and three connected business areas: retail sales, bicycle rentals, and bicycle service. Retail sales include bicycles, parts, and accessories sold both online and in stores, with inventory shared across locations. The rental operation manages approximately 400 bicycles, including e-bikes that provide telemetry information such as location and battery state. The service operation manages workshop appointments and repairs. A rental bicycle undergoing service is unavailable for rental, while a customer's bicycle being serviced occupies workshop capacity.

The current operations use separate software and information sources that do not provide consistent, real-time visibility across the business. Managers may not know whether inventory is available at another store, whether a rental bicycle is actually available, or how repairs affect rental availability and workshop capacity.

The proposed software will provide a shared view of important operational information without requiring Golden Bike Shop to replace every existing system. The system will help managers see inventory, rental fleet status, telemetry information, and service availability in one place.

If the project is not completed, the business risks inaccurate inventory information, unavailable rental bikes being shown as available, inefficient use of workshop capacity, missed sales or rental opportunities, and difficulty responding to seasonal demand and local events.

Stakeholders

Stakeholder	Role and Needs
Gregg Bishop	Owner/sponsor; approves budget, major functionality, and final completion. Needs business value while controlling cost.
Marcus Glover	Store manager and daily user; less technically experienced and needs a simple, understandable interface.
Rental/Fleet Staff	Need accurate rental status, location, battery information, and service availability.

Stakeholder	Role and Needs
Service/Workshop Staff	Need visibility into appointments, available bays, and bikes removed from rental service.
Retail/Rental Employees	Use operational information to assist customers and make day-to-day decisions.
Customers	Affected through improved inventory, rental availability, and visibility around bike service status.

The owner is the primary decision maker and budget authority. Store, rental, and service employees are the primary daily users. Customers are affected by the quality and accuracy of the information even though they may not directly use the system.

Potential conflicts include cost versus functionality, simplicity versus the number of features, and rental needs versus service needs. For example, managers may want more detailed information while the owner may prioritize keeping the project within budget and schedule.

Scope, Constraints, Assumptions, and Risks

In Scope

- Shared visibility of retail inventory across all four stores.
- Online and in-store inventory status.
- Product availability and store location.
- Rental fleet status for approximately 400 bikes.
- Rental availability, location, and service status.
- Telemetry information such as e-bike location and battery state.
- Visibility into workshop bookings and available capacity.
- Identification of rental bikes unavailable because they are being serviced.
- Role-appropriate views for managers and operational staff.

Out of Scope

- Replacing existing point-of-sale systems.
- Replacing rental payment systems.
- Installing or replacing telemetry hardware.

- Building a new consumer mobile application.
- Rebuilding the entire e-commerce storefront.
- Predictive artificial-intelligence demand forecasting.
- Completely replacing the existing workshop management system.
- Expansion beyond the four Golden Bike Shop locations during this project.

Assumptions

- The four stores will remain in operation during the project.
- The rental fleet will remain approximately 400 bikes.
- Existing systems can provide data through an available integration method.
- Existing telemetry hardware will remain operational.
- Store, rental, and service representatives will be available for requirements reviews and testing.
- The four-person development team will remain available for the eight-week project.
- Major hardware purchases and network construction are not required.
- The proposed system provides operational visibility rather than replacing every existing business system.

Constraints

The project must be completed within the planned project period and before the business's seasonal demand spike. The team has limited staffing and stakeholder availability. Information must remain sufficiently synchronized across four stores and multiple operational areas. The project also has a proposed $65,000 budget.

High-Level Risks

The largest risks are difficulty integrating existing systems, inaccurate or delayed telemetry, inconsistent inventory data, changing stakeholder requirements, limited manager availability, resistance to a new system, budget overruns, and incorrect definitions of what constitutes an "available" bicycle. These risks will be addressed through early requirements clarification, iterative reviews, prototypes, testing, and stakeholder acceptance.

Planned Schedule

Week	Milestone	Major Deliverable
1	Project Preparation	Charter, stakeholders, scope, process decision
2	Requirements	Initial requirements specification
3	Design	Interface and architecture designs
4	Rapid Prototype	Prototype and stakeholder feedback
5	Development Iteration 1	Retail inventory functionality
6	Development Iteration 2	Rental, telemetry, and service integration
7	Testing and UAT	Test results and user acceptance
8	Deployment Preparation	Training, final documentation, lessons learned

The schedule uses short deliverables and stakeholder reviews rather than waiting until the end of the project to demonstrate results. This approach provides opportunities to identify incorrect assumptions early.

Process Selection and Justification

Selected Approach: Evolutionary Lifecycle with Agile Practices

The team selected an evolutionary lifecycle with Agile practices for the remainder of the project. An evolutionary lifecycle is appropriate when requirements and customer needs may change as understanding increases and functionality can be introduced through successive iterations. Agile practices support frequent communication, stakeholder feedback, and incremental delivery. This project has uncertainty that makes an adaptive approach appropriate. The team must determine the availability retail inventory and rental bikes, how telemetry information should be presented, how service activity affects rental availability, and what information each manager actually needs. These details are difficult to establish before users see the proposed solution.

Alternatives Considered

Waterfall — Rejected.
Waterfall uses sequential phases and assumes that requirements can be established early and remain relatively stable. This project contains too much uncertainty around availability definitions, integrations, telemetry information, and user needs. The trade-off is that the team

gives up some upfront predictability in exchange for the ability to respond to discoveries during development.

Rapid Prototyping — Considered but Not Selected as the Primary Lifecycle.
Rapid prototyping emphasizes creating early prototypes to obtain feedback and improve understanding. It is valuable for this project, so the team will use rapid prototyping during Week 4. However, the project also requires requirements, architecture, integration, testing, training, and deployment planning. Therefore, rapid prototyping will be used as an Agile technique within the broader evolutionary approach rather than as the project's complete lifecycle.

What Could Change the Decision?

The team would reconsider its process if requirements became completely fixed, stakeholders became unavailable for iterative feedback, the sponsor required complete upfront approval before development, major regulatory requirements required a more predictive process, or the project changed from an evolving design exercise to a highly defined implementation with predetermined interfaces.

If uncertainty increased substantially, the team would increase the number of iterations and stakeholder reviews rather than immediately changing the overall lifecycle.

02 / REQUIREMENTS

Fourteen prioritized user stories and six measurable non-functional requirements turn the charter's scope into testable commitments, ranked by business value, operational risk and dependency.

System Scope

The system for Golden Bike Shop operates in three distinct domains: retail sales, bicycle rentals, and bicycle service.

Within the sales domain, key concepts are product availability, inventory counts, and location of inventory. The system will integrate with but will not replace existing point-of-sale and inventory requisition functionality. This part of the system will be built out to assist with day-to-day sales operations by keeping accurate and up-to-date records of how much of each product is on hand at any given location. That data will be used both to direct and track sales, and to facilitate leadership in ordering inventory.

Key concepts for rental bikes are type, availability, service status, location, and battery/telemetry status if applicable. The system will be built to replace manual record-keeping. Like with sales, existing functionality will be integrated with. For rentals, this means payments and telemetry data sourcing. This part of the system will assist with rentals in providing live data around bike availability and status both in-stores as well as in service and through e-bike reporting.

The service domain has key concepts of service bay status, calendar-based booking, bay count/capacity, and client or rental typing. The system will be built to streamline operations by reducing unused service bay time and reporting both the start and completion of bike service. While this will most directly affect the service staff, it will also greatly assist the day-to-day rental operations. There is an added layer here where customer bikes can be serviced. Added calendar booking functionality will help in maximizing bike service throughput while minimizing the amount of time customers have to go without their bikes. The system will include capacity for leadership to create and modify bookings, but beyond that, it will integrate with the existing service center workshop management process.

The three domains will be brought together into a shared view with role-based access. This gives management and the owner access to all the data needed to make quick and accurate decisions without manual overhead or codeswitching when going between apps/interfaces.

Functional Requirements

REQ-001: As a retail employee, I want to look up a product by UPC or name and see how much is available at my store, and at the other three locations, so that I can give customers accurate information about what we have in stock.

Acceptance Criteria:

- Typing a UPC into the search bar and pressing search pulls up product information.
- Typing a product name into the search bar and pressing search pulls up product information.
- Product information includes inventory counts at each of the four store locations.

This is the top priority because the core of the Golden Bike Shop's business is retail sales. In-store traffic accounts for around 70% of sales and sales make up 60% of the business' gross revenue. Employees need to be able to direct customers to in-stock products for sales to be made.

REQ-002: As a store manager, I want access to all inventory at my location so that I can place orders for low or out of stock products.

Acceptance Criteria:

- An authorized store manager can view a detailed breakdown of inventory at their store.
- The detailed view lists all valid products for that store in a table with columns for UPC, product name, on-hands, and historical sales counts.

This is the second highest priority because no sales can be made if items are not in stock. This requirement directly supports the one above, as well as the later online sales requirement.

REQ-003: As a rental employee, I want to update the status of a rental bike so that the system accurately reflects whether the bike is available, rented, or being serviced.

Acceptance Criteria:

- An authorized rental employee can update a bike's status.
- The available, rented, and servicing statuses are supported.
- The updated status is displayed when the bike is viewed or searched.
- Status changes are saved and remain available after the user leaves and returns to the bike record.

  Bike rentals make up 35% of the business' gross revenue. This requirement is necessary for all the automation and visibility the system will provide around rental bikes and directly enables the next two requirements.

REQ-004: As a rental employee, I want to rent bikes out to customers only when those bikes are available.

Acceptance Criteria:

- An authorized rental employee can rent a specific bike out to a customer when the bike's status is "available".
- The system denies a rental when the bike's status is "rented".
- The system denies a rental when the bike's status is "being serviced".

This is the requirement that facilitates the rental business. Once individual bikes can be assigned statuses, the system aids rental employees in renting out only available bikes. This has been a manual process up to this point, resulting in inconsistency and customer frustration when bikes are incorrectly put up for rent.

REQ-005: As a rental employee, I want to see bikes by status and type so that I can find an available bike that matches what a customer wants to rent.

Acceptance Criteria:

- Each rental bike has non-null type and status data.
- Type is either traditional or e-bike.
- Status is one of: available, rented, or servicing.
- Filtering bikes by type provides only results matching that type.
- Filtering bikes by status provides only results matching that status.
- Filtering bikes by both type and status provides only results matching both values.
- Status cannot be set to a customer bike status of "awaiting service", "in service", or "ready for pickup".

This requirement was placed just below the actual process of renting out bikes because it doesn't become useful before that work is in place. It is still a very high priority, though, because it streamlines the process of matching a customer with a rental bike.

REQ-006: As a lead service staff member, I want to manage service bookings so that I can maximize bay usage uptime.

Acceptance Criteria:

- An authorized lead service staff member can see a calendar view for each bay at the service center.

- Service appointments can be created, accepting a start time, end time, and bay number.
- The calendar view has visual distinction between blocked time for a bay and unblocked time for that bay.
- Existing appointments can be modified to change the start time, end time, or bay.
- Existing appointments can be deleted.
- Upon deletion, the calendar view refreshes and shows as unblocked between the start and end time of the appointment.
- Creating or changing an appointment checks for conflicts.
- If the appointment overlaps with an existing appointment, it fails to be created/updated and an error message shows.
- If the appointment doesn't overlap with an existing appointment, it is successfully created/updated.
- The calendar view refreshes and shows the correct start and end time in the given bay.

Rental bikes have a high margin for Golden Bike Shop because of their in-house servicing. To keep those margins high, the service shop needs to keep bay usage uptime high as well. This requirement is prioritized here because it creates the core functionality for the service center. This will also assist with high availability for individual rental bikes.

REQ-007: As a service staff member, I want to see all bookings for my bay for a given day so that I can optimize my time.

Acceptance Criteria:

- Logging in as a service staff member shows assigned bay number.
- Service staff members can access a calendar view for a given bay on a given day.
- For each booking in the calendar view, detailed information is available, including whether the booking is for a customer or rental bike.

This requirement supports the one above it. Giving service staff their own calendar view helps to ensure that they can get through all the assigned work within a day, even when they have multiple bookings.

REQ-008: As a customer, I want to order in-stock products online so that I can buy from the store without visiting a physical location.

Acceptance Criteria:

- Customers can view available products through the bike shop's website.
- When viewing a product, it is shown as either in stock or out of stock.
- This data is pulled from the same source as in-store inventory.

- An item will show as out-of-stock if it has zero inventory at all four locations.
- The system denies attempts to purchase when an item is out of stock.
- The system allows attempts to purchase when an item is in-stock and directs customers to existing payment processing systems.

Online traffic accounts for 30% of all sales for the shop. This requirement helps to ensure that online in-stocks are accurate and reduce customer frustration when ordering products.

REQ-009: As a store manager, I want to pull up historical sales data so that I can monitor sales trends and compare store performance over time.

Acceptance Criteria:

- Managers can view historical sales data.
- Managers can filter sales data by date, time, product, and who it was sold by.
- Detailed data can be pulled up to two years in the past.
- Aggregated data can be pulled up to ten years in the past.

Full trend forecasting, especially with AI assistance, is not part of our project scope. However, this requirement is still very useful in facilitating basic forecasting to keep products in stock.

REQ-010: As a service staff member, I want to update the service status of a customer's bike after it is received by the shop, so that staff can track the bike's progress through the service process and provide accurate status information.

Acceptance Criteria:

- Service staff can select a customer bike associated with a service appointment.
- Service staff can update the bike's status to one of the defined statuses:
- Awaiting Service
- In Service
- Ready for Pickup
- The current service status is displayed with the customer's bike ID and assigned service bay on the Service screen.
- The system saves the updated status and associates it with the correct customer bike and service appointment.
- Authorized staff can view the current service status of a customer bike.
- These status fields are only used for customer bikes, and customer bikes cannot be set to the rental bike statuses of "available", "rented", or "being serviced".

Customer bike service makes up only five percent of the business' gross revenue, so requirements specific to it are prioritized low. Customer bike service can be done without this requirement, but having it implemented helps with visibility for both employees that interact with those customers and the customers themselves.

REQ-011: As a customer, I want to check the status of my bike service so that I know what is happening to it and when I can pick it up.

Acceptance Criteria:

- An authorized customer can associate bikes with their account.
- They can select a bike within their account to see its service status.

The previous requirement allows staff to inform customers about the status of their bike service. This requirement allows them to view it themselves, which helps with customer trust. Since word-of-mouth is used significantly around building the customer bike service portion of Golden Bike Shop's business, building trust in this way is still important.

REQ-012: As a rental employee, I want to pull up e-bike reporting data so that I can monitor for health and proper usage.

Acceptance Criteria:

- When viewing a rental bike with type e-bike, reporting data is shown.
- Reporting data is timestamped.
- Battery level, mileage, and location data are available.
- Low battery or other negative reports are flagged within the system and show the bike as unhealthy.

This requirement is low priority because it doesn't connect rental employees with new data. They can already use existing systems to access telemetry data. This requirement is what pulls it into the unified system to increase ease and visibility of telemetry data.

REQ-013: As a rental manager, I want to pull up historical rental data so that I can verify past rentals, resolve customer questions, and analyze rental trends.

Acceptance Criteria:

- The rental manager can view completed and past rental transactions.
- The rental manager can search or filter rental history by customer, rental item, or date range.

- The rental manager can open a past rental to view details such as rental dates, item, customer, status, and total cost.

This is our lowest priority since historical rental data doesn't have any direct mapping onto actions. Unlike with sales, there is no inventory to order. Other requirements have also been written to keep rental bike availability maximized. The owner of the bike shop has still indicated that he would like access to this data.

REQ-014: As a customer, I want to reserve a bicycle for a future time period so that I can confirm the bicycle I want will be available.

Acceptance Criteria:

- An authorized customer can request a start date and time, end date and time, location, and specific rental bicycle.
- The system accepts a reservation only when the bicycle is eligible for rental at the specified location and has no conflicting rental or reservation already booked.
- Conflicting reservation attempts are rejected without creating a duplicate record.
- A valid reservation is accepted, persisted, and viewable.
- An existing reservation can be cancelled, which frees the reserved date/time interval.
- Rental checkout rechecks current bike status, so a reservation never overrides a bike that is currently rented or servicing.

Non-Functional Requirements

NFR-001: Changes to operational data (inventory on-hand counts, rental bike status, reservations and service appointments) shall be committed to the database and visible to all users within 2 seconds of the transaction.

Acceptance Criteria:

- A purchase reduces that item's on-hand count at the store where it was made, and the change is visible within 2 seconds.
- A rental bike status change is visible to all users within 2 seconds.
- A new, changed or cancelled reservation or service appointment is reflected within 2 seconds.
- Reports and historical queries are excluded and fall under NFR-006's response-time monitoring.

NFR-002: The system shall operate correctly on the current versions of the organization-approved web browsers used at all four Golden Bike Shop locations without requiring installation of specialized client software.

Acceptance Criteria:

- The system successfully operates in each organization-approved browser.
- Core functions work correctly across the supported browsers, including viewing inventory, rental status, telemetry information, and service availability.
- Users do not need to install specialized software on their computers to access the system.
- The interface remains usable and functional across the supported browser environments.

NFR-003: The system shall enforce role-based access control so that users can access only the functions and information appropriate to their assigned role, with manager-level functions restricted to authorized managers and administrators.

Acceptance Criteria:

- A manager can access manager-level functions that are not available to regular employees.
- A regular employee cannot access manager-level functions.
- Users can access the operational information required for their assigned role.
- Unauthorized attempts to access restricted functions are denied.
- User roles and permissions can be verified during testing.

NFR-004: The system shall include sufficient technical documentation and code organization to allow a developer familiar with the project to understand and modify individual system components without requiring extensive assistance from the original developer.

Acceptance Criteria:

- Major system components are clearly identified and documented.
- The code includes comments or documentation where the purpose or operation of a component is not immediately apparent.
- Setup and deployment instructions are documented.
- A developer who did not originally create a component can identify where changes to that component should be made.
- A developer familiar with the project can make a minor change to an individual component using the project documentation without requiring assistance from the original developer.

NFR-005: The system shall provide a clear and consistent user interface that allows trained store, rental, and service employees to perform their primary tasks without requiring technical assistance.

Acceptance Criteria:

- At least 80% of test users can complete their assigned primary tasks without assistance.
- Users can navigate to the information required for their role.
- Interface labels and navigation are consistent throughout the application.
- Any usability issues identified during testing are documented and addressed before deployment when practical.

NFR-006: The system shall be available to authorized users at least 99.5% of the time during normal business operating hours, excluding scheduled maintenance that is communicated to users in advance.

Acceptance Criteria:

- The system alerts on any response time exceeding 3 seconds.
- The system is down for no more than 3 minutes total in Golden Bike Shop's normal operating hours of 9am to 7pm.
- The system displays a toast on log-in to inform users of scheduled maintenance when applicable.

Project Charter Changes

When developing the functional requirements, we realized that our specified roles in the project charter were too limited. For example, it made the most sense to have a dedicated lead service staff role to be in charge of creating and modifying bookings. We separated out management positions for retail and service staff to support distinctions between their usage of the system and the usage of base-level employees.

We also determined as part of this work that it would be important for customers to have some visibility into the status of their bikes when they have left them with the shop for service. The charter only had customers listed as indirectly affected by changes to the system, but this was one instance where they would be directly affected.

03 / USER JOURNEYS & INTERFACE

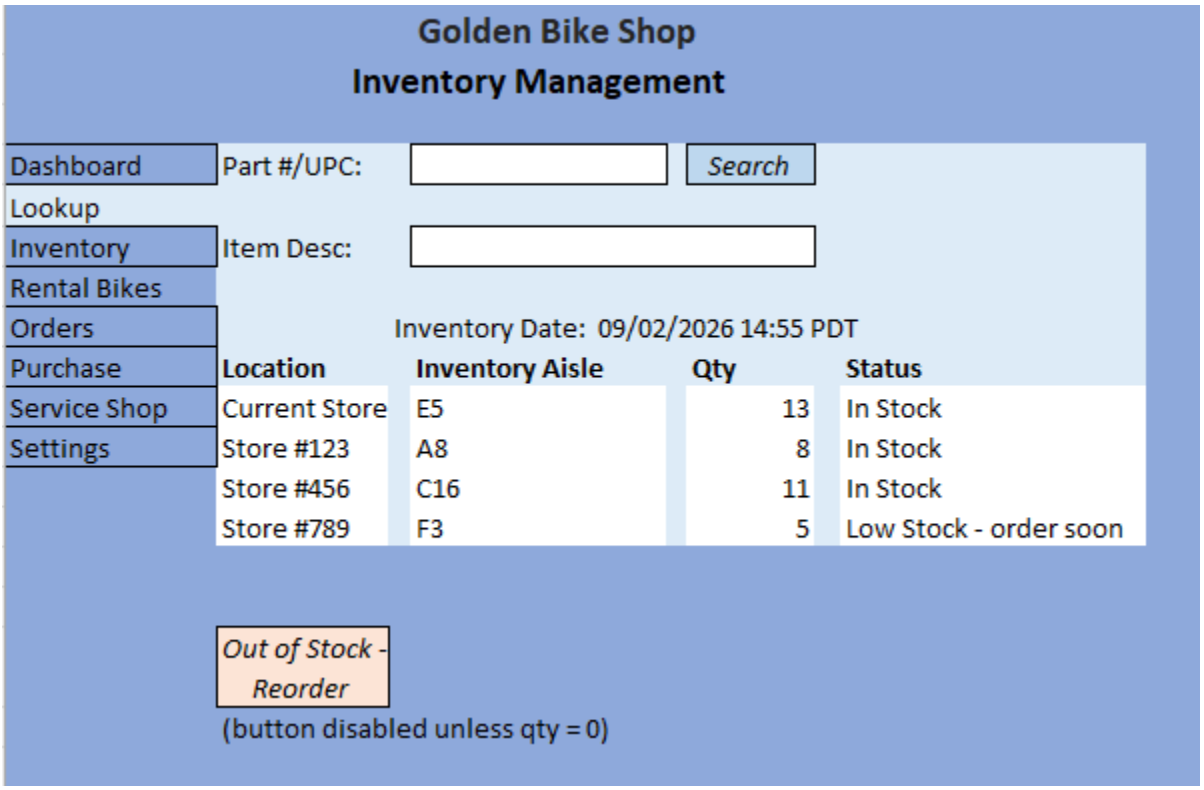
Three end-to-end journeys, for the retail manager, rental employee and lead service staff, show how the highest-priority requirements become simple, accessible screens, with every step traced back to a requirement.

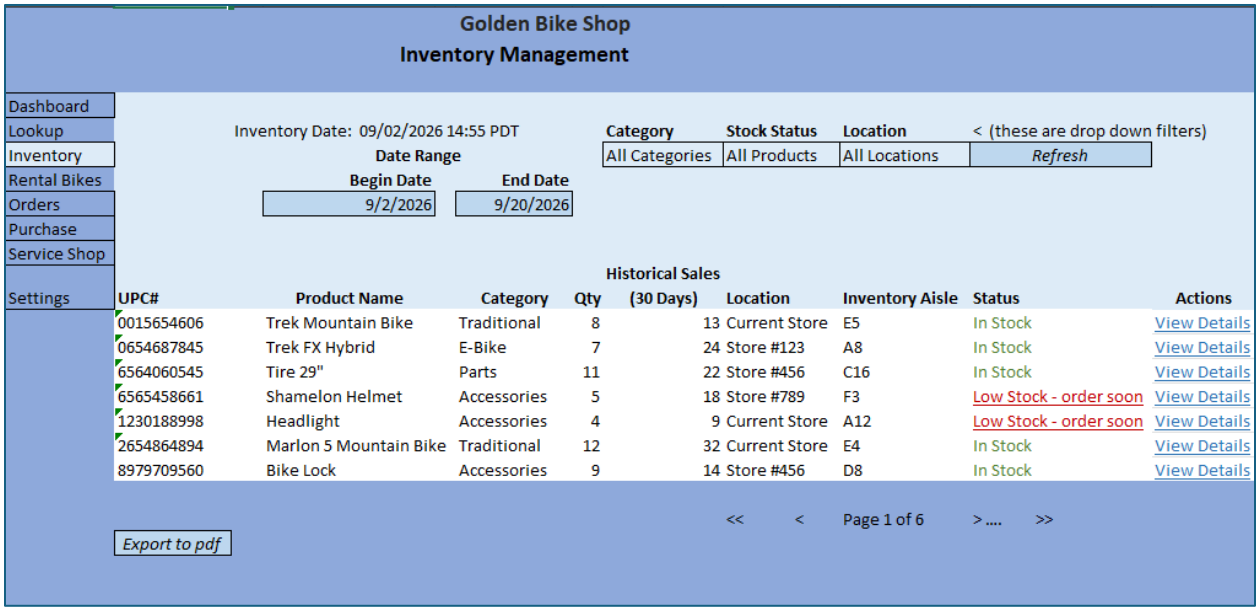
User Interface Designs

Journey 1 — **Retail Manager Manages Product Availability and Inventory (REQ-001 + REQ-002 + REQ-009)**

User*:* Retail Manager

Goal*:* Provide accurate inventory information to customers and manage stock levels by reviewing detailed inventory and historical sales data.





Golden Bike Shop
Inventory Management

Dashboard
Lookup
Inventory
Rental Bikes
Orders
Purchase
Service Shop
Settings

Inventory Date: 09/02/2026 14:55 PDT

Date Range
Begin Date: 9/2/2026
End Date: 9/20/2026

Category: All Categories
Stock Status: All Products
Location: All Locations
< (these are drop down filters)
Refresh

UPC#	Product Name	Category	Qty	Historical Sales (30 Days)	Location	Inventory Aisle	Status	Actions
0015654606	Trek Mountain Bike	Traditional	8	13	Current Store	E5	In Stock	View Details
0654687845	Trek FX Hybrid	E-Bike	7	24	Store #123	A8	In Stock	View Details
6564060545	Tire 29"	Parts	11	22	Store #456	C16	In Stock	View Details
6565458661	Shamelon Helmet	Accessories	5	18	Store #789	F3	Low Stock - order soon	View Details
1230188998	Headlight	Accessories	4	9	Current Store	A12	Low Stock - order soon	View Details
2654864894	Marlon 5 Mountain Bike	Traditional	12	32	Current Store	E4	In Stock	View Details
8979709560	Bike Lock	Accessories	9	14	Store #456	D8	In Stock	View Details

<< < Page 1 of 6 > .... >>

Export to pdf

Steps, System Actions, and Failure Points

1. Manager searches for a product by UPC

- **System:** Validates UPC → queries unified inventory database → returns product record.
- **Failure points:**
  - Invalid UPC
  - Inventory sync delay (>2 seconds)
  - Network or database outage

2. Manager searches for a product by name

- **System:** Performs fuzzy name matching → returns list of matching products → manager selects correct item.
- **Failure points:**
  - Misspellings or ambiguous names
  - Slow search response
  - Incorrect or incomplete product metadata

3. Manager views inventory counts across all four locations

- **System:** Displays real-time on-hand counts for each store location.
- **Failure points:**
  - Stale inventory data
  - Missing location data
  - UI confusion around multi-location counts

4. Manager opens detailed inventory breakdown for their store

- **System:** Shows table with UPC, product name, on-hands, and historical sales counts.
- **Failure points:**
  - Incorrect historical sales mapping
  - Missing product entries
  - Role-based access misconfiguration

5. Manager reviews historical sales data

- **System:** Allows filtering by date, time, product, and employee.
- **Failure points:**
  - Slow query performance
  - Incorrect filtering logic

    - Missing or incomplete sales records

6. Manager decides whether to reorder low-stock items

- **System:** Provides inventory + sales trends → manager uses this to determine reorder needs.
- **Failure points:**
    - Misinterpreted data
    - Outdated counts due to sync latency
    - Missing reorder indicators

7. Manager communicates availability or stock decisions to staff

- **System**: Ensures inventory data is consistent across retail, rental, and online systems.
- **Failure points**:
    - Staff misunderstanding
    - UI inconsistencies
    - Inventory changes not propagating to online storefront

Why This Journey Matters Most

- It supports the highest-priority requirement in the entire system.
- Retail sales account for 60% of gross revenue and 70% of in-store traffic.
- Managers are responsible for keeping products in stock, which directly affects revenue.
- This journey touches inventory accuracy, role-based access, historical sales, and multi-location data, making it foundational for the business.

Primary Requirements

- **REQ-001** — *Retail employee/manager looks up product by UPC or name and sees inventory at all four locations.* This covers searching, viewing multi-location inventory, and providing accurate stock information.
- **REQ-002** — *Store manager views detailed inventory breakdown for their location.* This includes UPC, product name, on-hands, and historical sales counts.
- **REQ-009** — *Store manager views historical sales data.* This supports trend monitoring and reorder decisions.

Supporting Non-Functional Requirements

These apply to the journey because they govern how inventory and manager-level access must work:

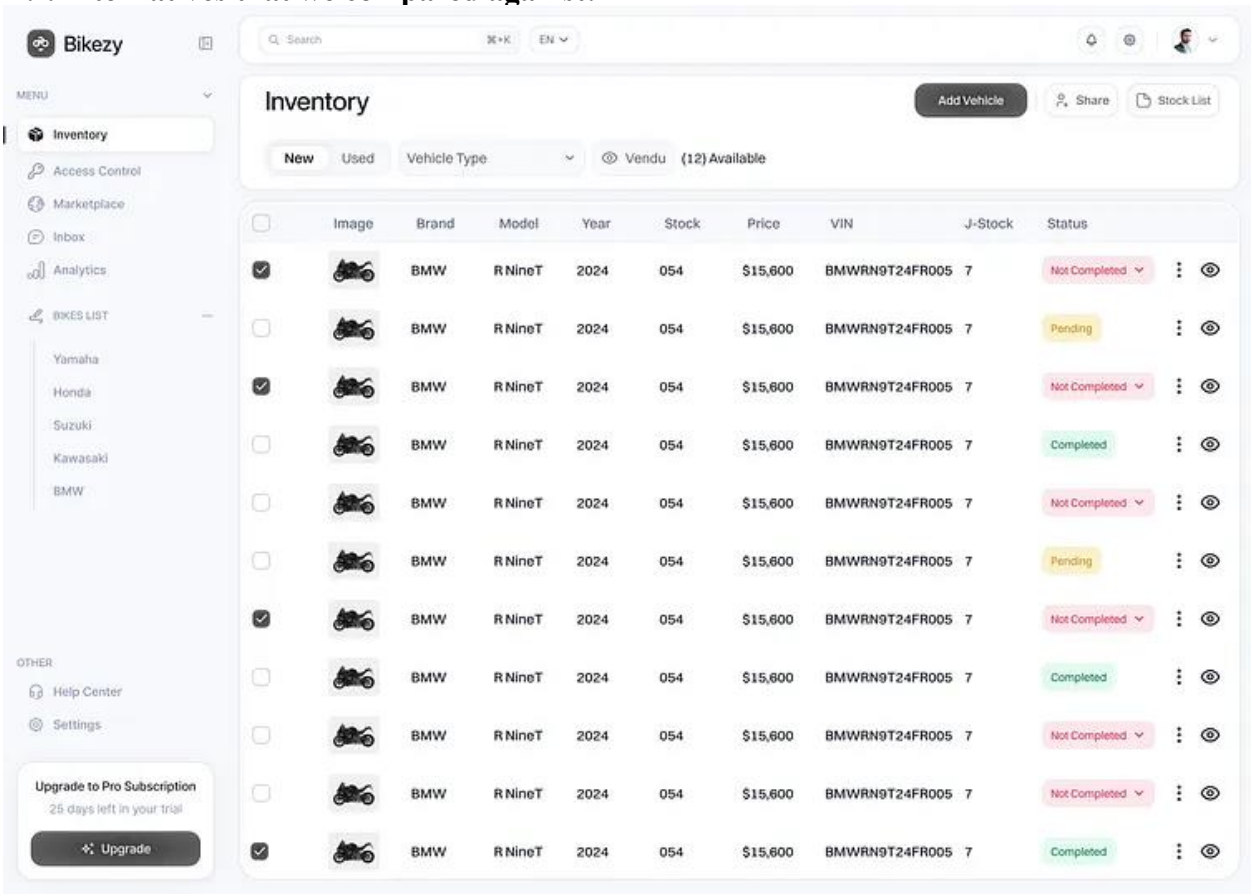
- **NFR-001** — *2 second operational data change SLA.* Critical for accurate multi-location inventory lookup.
- **NFR-003** — *Role-based access control.* Ensures managers can access manager-level inventory functions while employees cannot.
- **NFR-005** — *Clear and consistent user interface.* Supports the manager's ability to navigate inventory, sales history, and product lookup without assistance.

Journey Step	Requirement(s)
Search by UPC	REQ-001
Search by product name	REQ-001
View inventory across all locations	REQ-001**,** NFR-001
Open detailed inventory breakdown	REQ-002**,** NFR-003
View historical sales data	REQ-009
Make reorder decisions	REQ-002**,** REQ-009

Design Reasoning

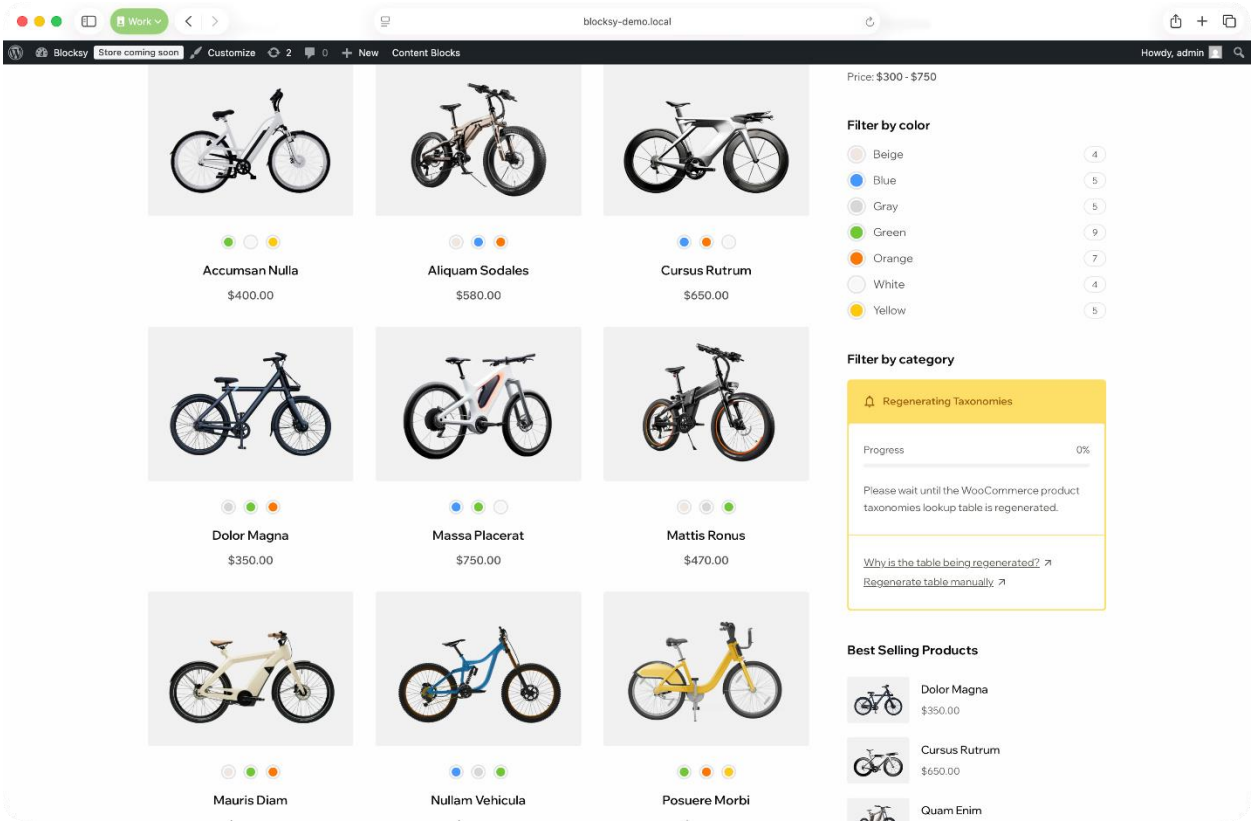
- **Ease of use:** The interface makes inventory lookup and reorder decisions fast by combining search, multi-location counts, and reorder indicators on one screen.
- **Applied principles:**
    - **Separation of Concerns:** Lookup and reorder are distinct, each with its own interface and logic, reducing cognitive load.
    - **Abstraction:** The manager sees only relevant data (UPC, stock, trends) while implementation details like database queries remain hidden.
    - **Uniformity:** Consistent table layouts and navigation menus across all store modules reinforce predictability.
    - **Verifiability:** Each action (search, reorder) produces visible confirmation, allowing managers to verify success.
- **Accessibility:**
    - Color indicators are paired with text ("Out of Stock – Reorder") for color-blind users.
    - Keyboard navigation supports all controls.
    - Text contrast and size are consistent throughout
    - Error messages specify corrective action ("Enter a valid UPC").

A.I. Alternatives that we compared against:



What they did differently was they included way too many details, such as an icon image next to the inventory item, and the details listed all on the Main Inventory page. We like that we included a link to "View Details" as a separate module so that the users can see the main inventory information necessary, then bring up details if needed, and this revealed that we were trying to keep the overall interface simple

to avoid cognitive overload and too many items all over the page. We kept the statuses color coded to identify the statuses easier and saw that the A.I. screen created the same similarity.



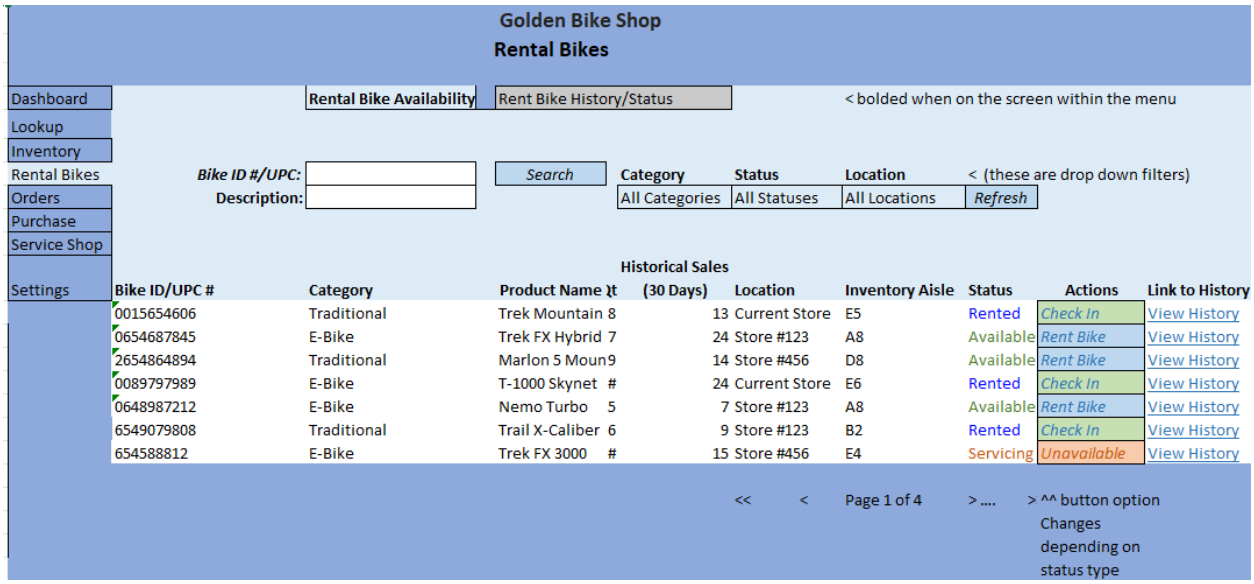
What they did differently was they listed large icons above each inventory bike item, making it easier to see which bike type and model you're looking at. We are keeping our original design layout for simplicity while still covering all of the necessary functionality and information.

Journey 2 — Rental Employee Rents Out a Bike (REQ-003 + REQ-004 + REQ-005)

User: Rental Employee

Goal: Rent a bike only when it is available and appropriate for the customer

Rental Bike Status Update Screen



Golden Bike Shop
Rental Bikes

Dashboard | Lookup | Inventory | Rental Bikes | Orders | Purchase | Service Shop | Settings

Rental Bike Availability | Rent Bike History/Status

< bolded when on the screen within the menu

Bike ID #/UPC: | Description: | Search

Category: All Categories | Status: All Statuses | Location: All Locations | Refresh

< (these are drop down filters)

Bike ID/UPC #	Category	Product Name	Historical Sales (30 Days)	Location	Inventory Aisle	Status	Actions	Link to History
0015654606	Traditional	Trek Mountain 8	13	Current Store	E5	Rented	Check In	View History
0654687845	E-Bike	Trek FX Hybrid 7	24	Store #123	A8	Available	Rent Bike	View History
2654864894	Traditional	Marlon 5 Moun 9	14	Store #456	D8	Available	Rent Bike	View History
0089797989	E-Bike	T-1000 Skynet #	24	Current Store	E6	Rented	Check In	View History
0648987212	E-Bike	Nemo Turbo 5	7	Store #123	A8	Available	Rent Bike	View History
6549079808	Traditional	Trail X-Caliber 6	9	Store #123	B2	Rented	Check In	View History
654588812	E-Bike	Trek FX 3000 #	15	Store #456	E4	Servicing	Unavailable	View History

<< < Page 1 of 4 > ....

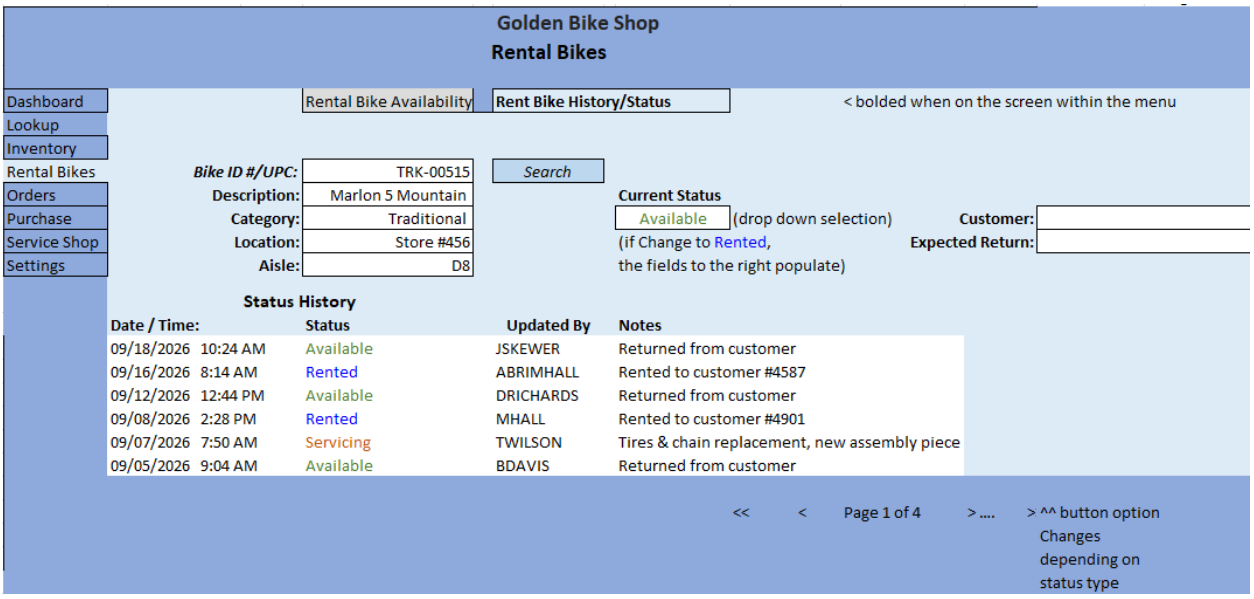
> ^^ button option Changes depending on status type

Steps, System Actions, and Failure Points

1. **Employee views bike list**
   - *System:* Loads all rental bikes with type + status.
   - *Where it can fail:* Missing type/status, slow load, incorrect filtering.
2. **Employee filters by type or status**
   - *System:* Returns only bikes matching "available" + chosen type (traditional/e-bike).
   - *Where it can fail:* Incorrect filter logic, null type/status fields.
3. **Employee selects a bike**
   - *System:* Shows bike details, telemetry (if e-bike), and current status.
   - *Where it can fail:* Telemetry unavailable, stale status, bike incorrectly marked available.
4. **Employee attempts rental**
   - *System:* Checks status → allows rental only if "available."
   - *Where it can fail:* Status not updated, race condition (two employees renting same bike), service bay conflict.
5. **System confirms rental**
   - *System:* Updates status to "rented," logs transaction, syncs data.
   - *Where it can fail:* Sync delay, failed status update, payment integration issues.

Why This Journey Matters Most

- Rentals make up **35% of gross revenue**.
- Incorrect rentals cause **customer frustration** and **operational risk**.
- This journey ties together **three requirements**: status updates, rental validation, and filtering.



Primary Functional Requirements

- **REQ-003** — Rental employee updates bike status (available, rented, servicing).
- **REQ-004** — System only allows rentals when bike status = "available"; denies rentals otherwise.
- **REQ-005** — Bikes must have non-null type + status; filtering by type/status must work.
- **REQ-012** — E-bike telemetry reporting (battery, location, health flags) shown when viewing an e-bike.

Supporting Non-Functional Requirements

- **NFR-001** — Status data must sync within 2 seconds.
- **NFR-003** — Role-based access control (rental employees can rent bikes; others cannot).
- **NFR-005** — Clear and consistent UI enabling employees to complete tasks without assistance.

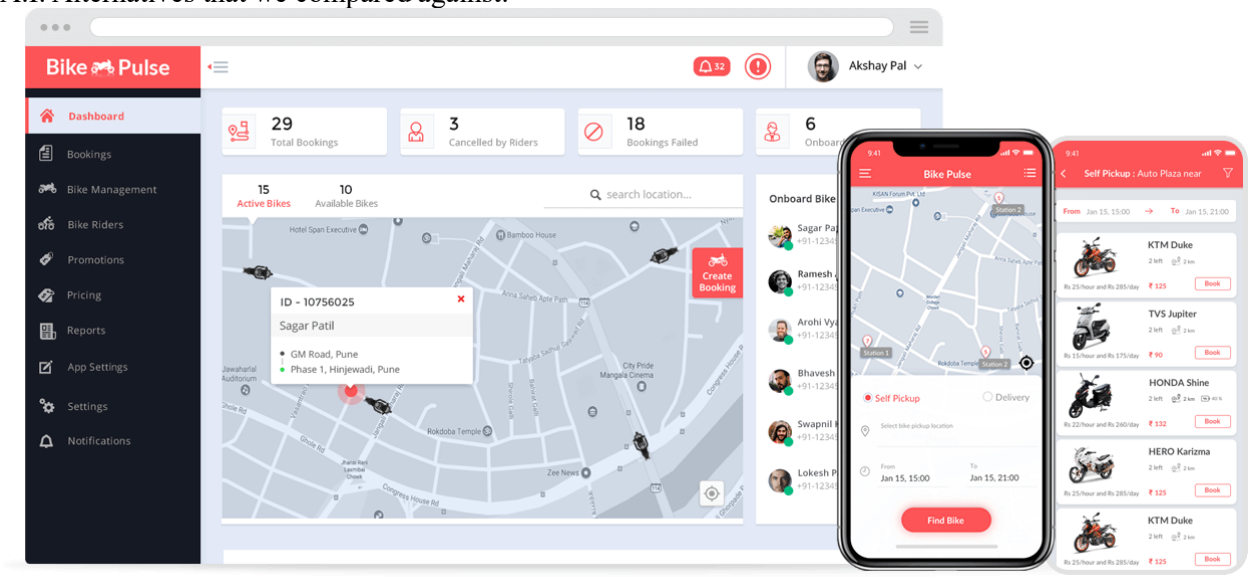
Journey Step	Requirement(s)
View rental bike list	REQ-003, REQ-005
Filter bikes by type or status	REQ-005
Select a bike and view details	REQ-003, REQ-005, REQ-012
Attempt to rent a bike	REQ-004, NFR-003
System confirms or denies rental	REQ-004, NFR-003, NFR-001
System updates bike status to "rented"	REQ-003, NFR-001

Design Reasoning

- **Ease of use:** Employees can see availability, rent, or check-in bikes from one screen, minimizing switching between modules.

- **Trade-off:** Traded off adding an additional screen because it would have been too convoluted.
- **Applied principles:**
    - **Modularization:** Bike status and rental actions are separate components that interact through defined interfaces.
    - **Uniformity:** Status labels and icons follow the same pattern across all bike types.
    - **Completeness:** The screen captures all essential data (type, status, battery, mileage) needed for a rental decision.
- **Accessibility:**
    - Status colors (green/red) are reinforced with text.
    - Keyboard shortcuts allow quick filtering.
    - Screen-reader labels identify each bike row and action.
    - Error messages guide correction ("Bike is currently rented; select another available bike").

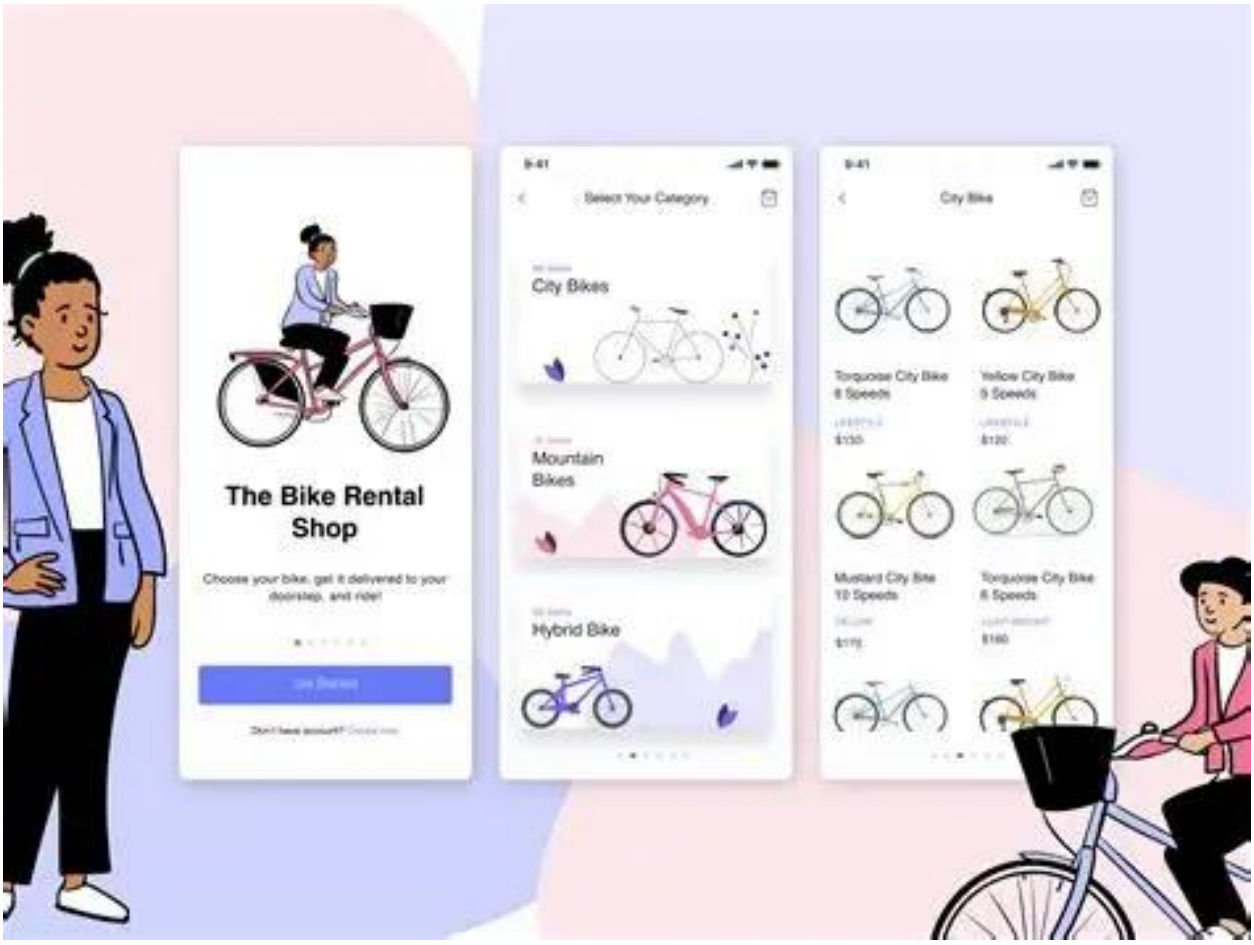
A.I. Alternatives that we compared against:



Admin Panel

Riders App

What they did differently was provide a dashboard overview on a map to view what appears to be multiple bikes in the area with a hover over a bike to view its status. We chose not to go with this design even though it looks like it would be a useful tool to track down rental bikes by exact location, but we felt this detail was not needed for the basic needs of the stakeholders.



This was the second image provided by A.I. for the rental bikes. What they did differently was they used large icons next to each inventory bike item, making it easier to see which bike type and model you're looking at. We are keeping our original design layout to avoid uploading individual images for each bike item.

Journey 3 — **Lead Service Staff Manages Service Bay Bookings (REQ-006 + REQ-007)**

User: Lead Service Staff

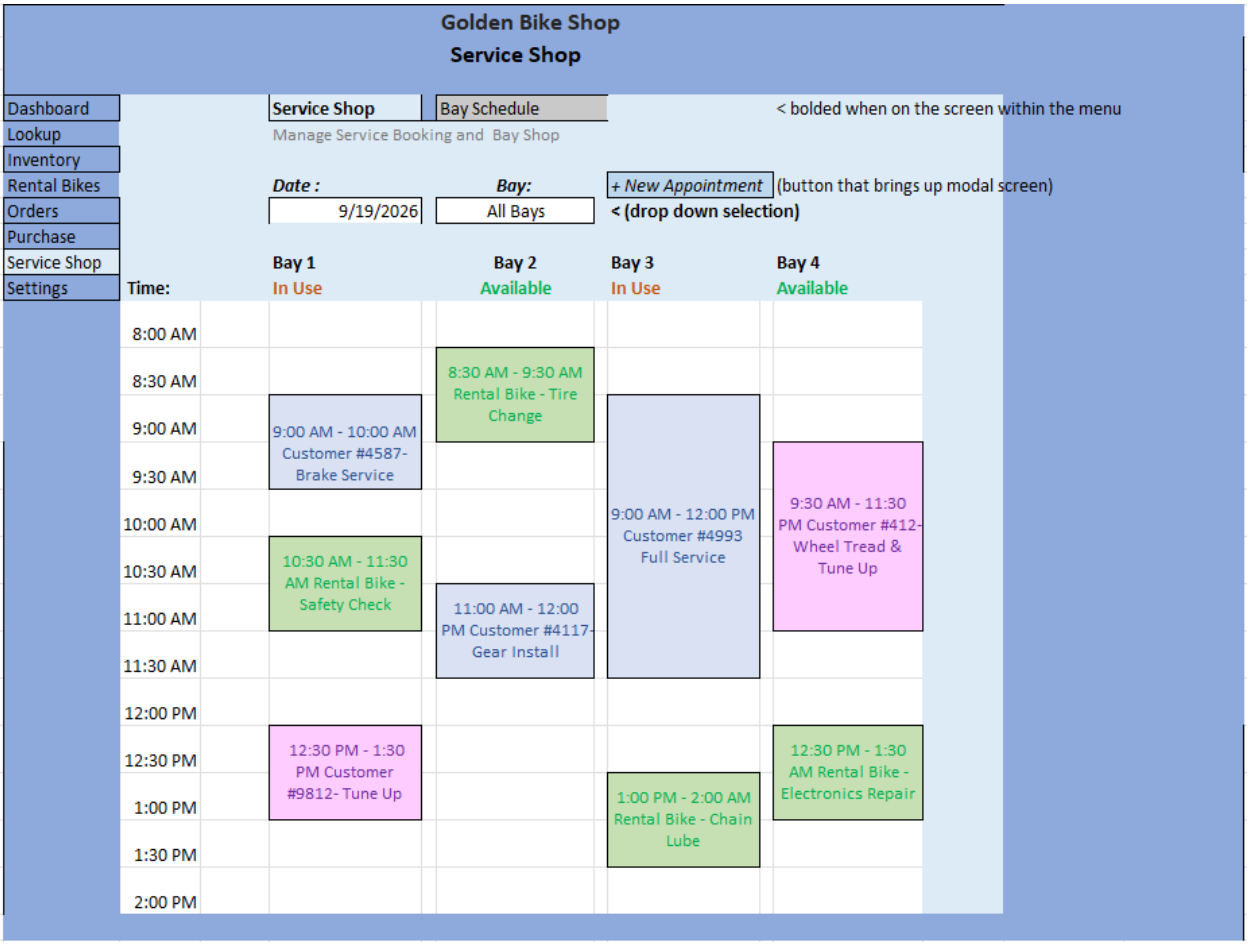
Goal: Maximize service bay uptime and avoid booking conflicts

Steps, System Actions, and Failure Points

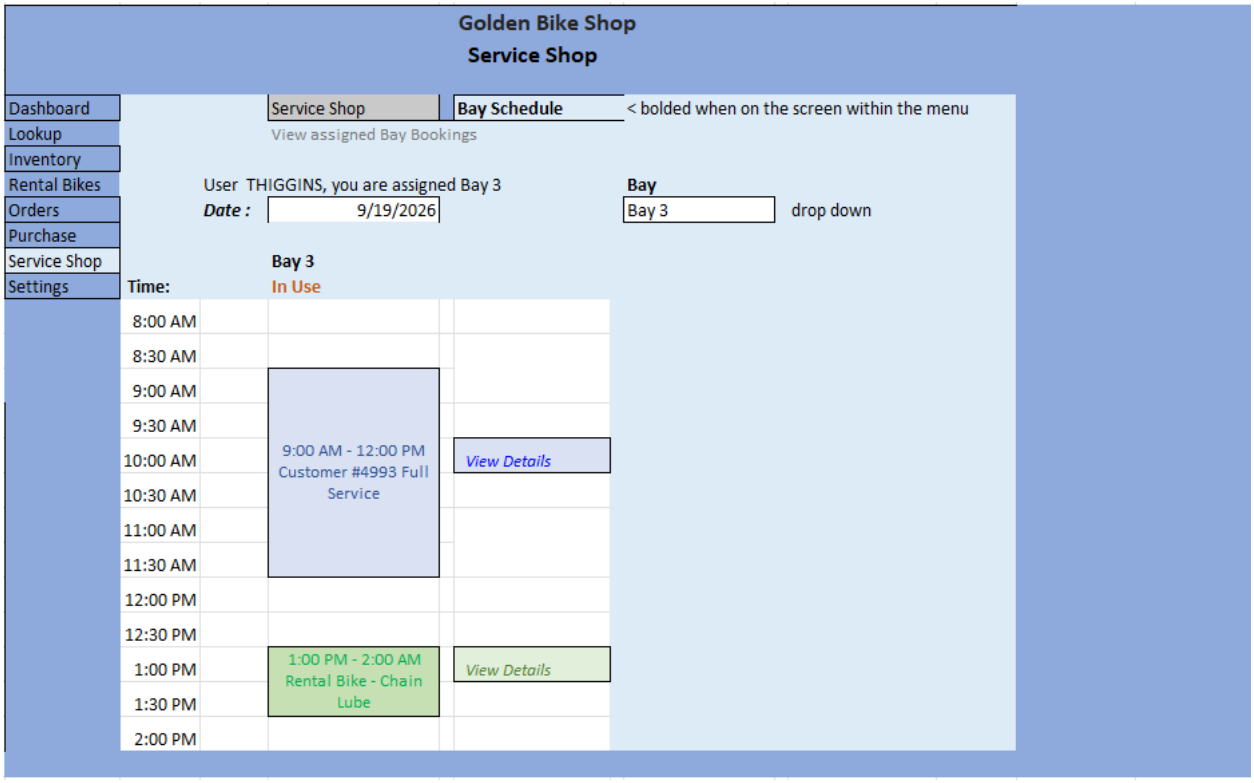
1. **Staff opens calendar view**
    - *System:* Shows all bays with blocked/unblocked time.
    - *Where it can fail:* Incorrect bay assignments, missing appointments, UI rendering issues.
2. **Staff creates a booking**
    - *System:* Accepts start/end time + bay → checks for conflicts.
    - *Where it can fail:* Overlapping appointments, incorrect conflict detection, invalid time ranges.
3. **System confirms or denies booking**
    - *System:* If no conflict → creates appointment; else → shows error.
    - *Where it can fail:* False positives/negatives in conflict logic, slow refresh.
4. **Staff modifies or deletes booking**
    - *System:* Updates calendar and re-checks conflicts.
    - *Where it can fail:* Calendar not refreshing, appointment not saved, incorrect bay mapping.
5. **Service staff views daily bay schedule**
    - *System:* Shows all bookings for assigned bay with customer/rental bike details.
    - *Where it can fail:* Wrong bay assignment, missing details, UI confusion.

Why This Journey Matters Most

- Service bay uptime directly affects **rental bike availability**, which affects **35% of revenue**.
- Poor scheduling leads to **unused bay time**, **delays**, and **customer dissatisfaction**.
- This journey supports both **service operations** and **rental operations**, making it a cross-domain priority.



Service Staff Daily Bay View Screen



Primary Functional Requirements

- **REQ-006** — Lead service staff manages service bay bookings (calendar view, create, modify, delete, conflict detection).
- **REQ-007** — Service staff view daily bay schedule with booking details (customer vs rental bike).

Supporting Non-Functional Requirements

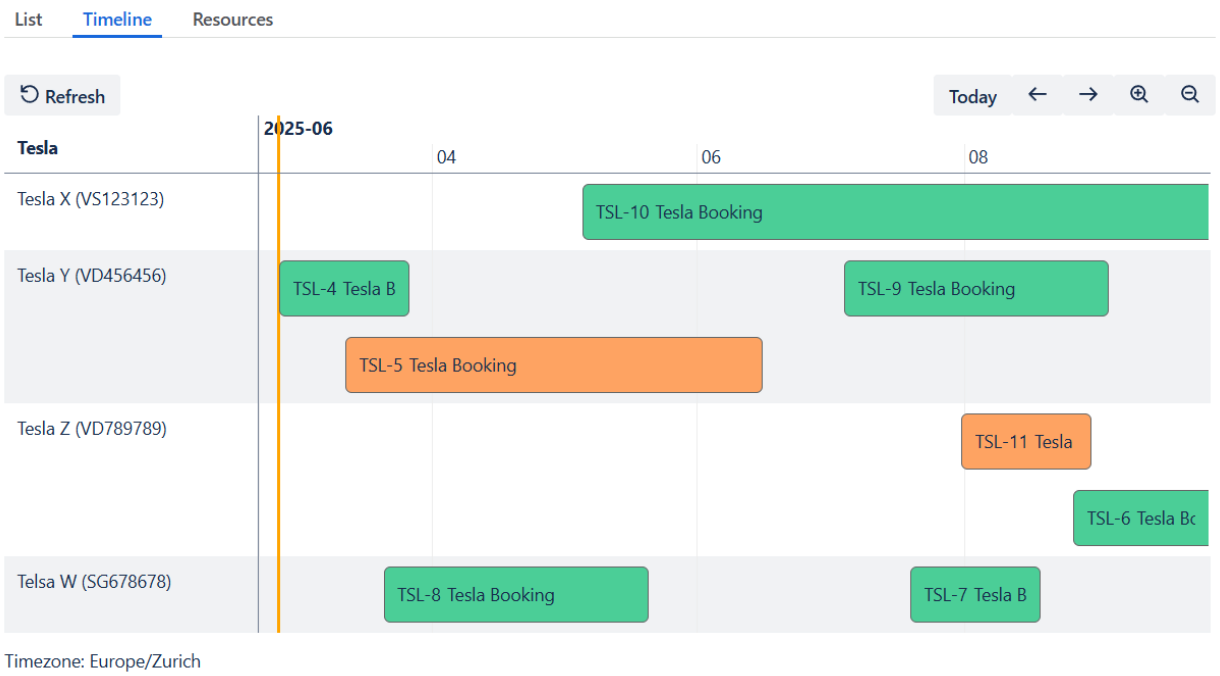
- **NFR-003** — Role-based access control (lead service staff vs regular service staff).
- **NFR-005** — Clear and consistent UI enabling staff to navigate bays, bookings, and details.

Journey Step	Requirement(s)
Open calendar view for all bays	REQ-006, NFR-003
Create a service appointment	REQ-006
System checks for conflicts	REQ-006
Modify or delete an appointment	REQ-006
Service staff views daily bay schedule	REQ-007, NFR-005
Service staff optimizes daily workflow	REQ-007, NFR-005

Design Reasoning

- **Ease of use:** The calendar view lets staff see bay availability at a glance and create bookings directly.
- **Trade-off:** Went for simplicity to see all bays with appointments displayed on one screen.
- **Applied principles:**
    - **Separation of Interface and Implementation:** The calendar UI exposes only booking controls; scheduling logic runs behind the scenes with the New Appointment button.
    - **Cohesion:** Each bay calendar handles only its own appointments, improving clarity and maintainability.
    - **Coupling:** Conflict-checking logic is independent of the visual calendar, enabling updates without UI redesign.
    - **Uniformity:** All bays use identical color-coded blocks and time formats.
    - **Completeness and Verifiability:** The system ensures every booking includes start/end time, bay number, and conflict validation, making results verifiable.
- **Accessibility:**
    - Color coding is paired with text (“In Use,” “Available”).
    - Keyboard navigation supports all appointment actions.
    - Time slots use readable 12-hour formatting.
    - Error messages explain next steps
      (“Selected time conflicts with Customer #4993 Full Service; choose another slot”).

A.I. Alternatives that we compared against:



What they did differently was listed out the appt bookings horizontally instead of vertically. We're not sure why A.I. produced an image for Teslas. Unless Tesla also produces E-bikes, but the A.I. was given the Golden Bike shop information with requirements and constraints. We chose not to go with this design because our layout seemed to make more sense and was visually more appealing overall.



This was the second image provided by A.I. for the Service/Bay screen. What they did differently was include serial numbers, which actually might be helpful when recording service information. We may list those details in our View Details link to a modal window. They included the servicing details on this screen instead of the scheduling, which is where our focus was. We are keeping our original design layout with the View Details link that will capture the necessary servicing details.

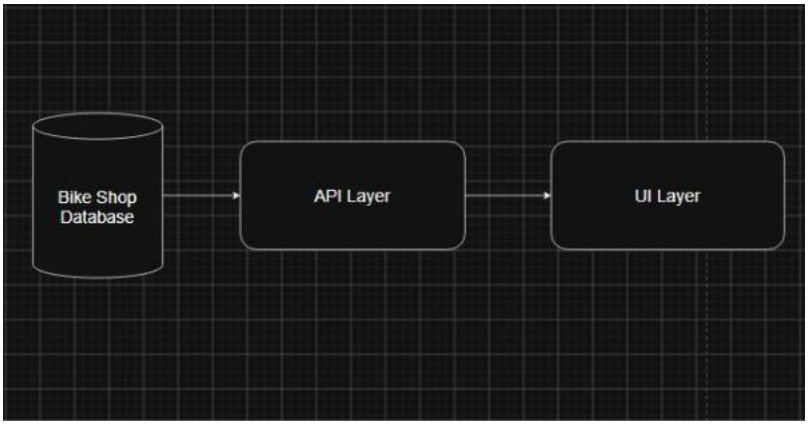
04 / APPLICATION ARCHITECTURE

The selected N-tier architecture uses a unified presentation, business-logic and data arrangement, logically separated services, and availability safeguards.

Architectural Approach

For the Golden Bike Shop, we chose to do an N-Tier approach to the application. The requirements themselves assumed a unified database, API Layer, and Presentation Layer (UI). Each layer would serve its own purpose. For the database layer, it would be persistent storage for the bikes, customers, telemetry data, orders, authentication, and other related information. The API layer would hold the business rules for rentals, reporting, sales, customers, etc. And lastly the UI layer would hold the interface that would be used by customers, store staff, and managers.

An n-tier monolithic backend reduces deployment and operational complexity because the application can be deployed as a single backend. The trade-off is increased coupling between business areas compared with independently deployable microservices. The team determined that the simpler deployment model was more valuable for the current size of the Golden Bike Shop system. The volume of traffic the system must support is served just as well with the simpler model without the need for added complexity, and the overlap between the various parts of the system means that some coupling is already expected.



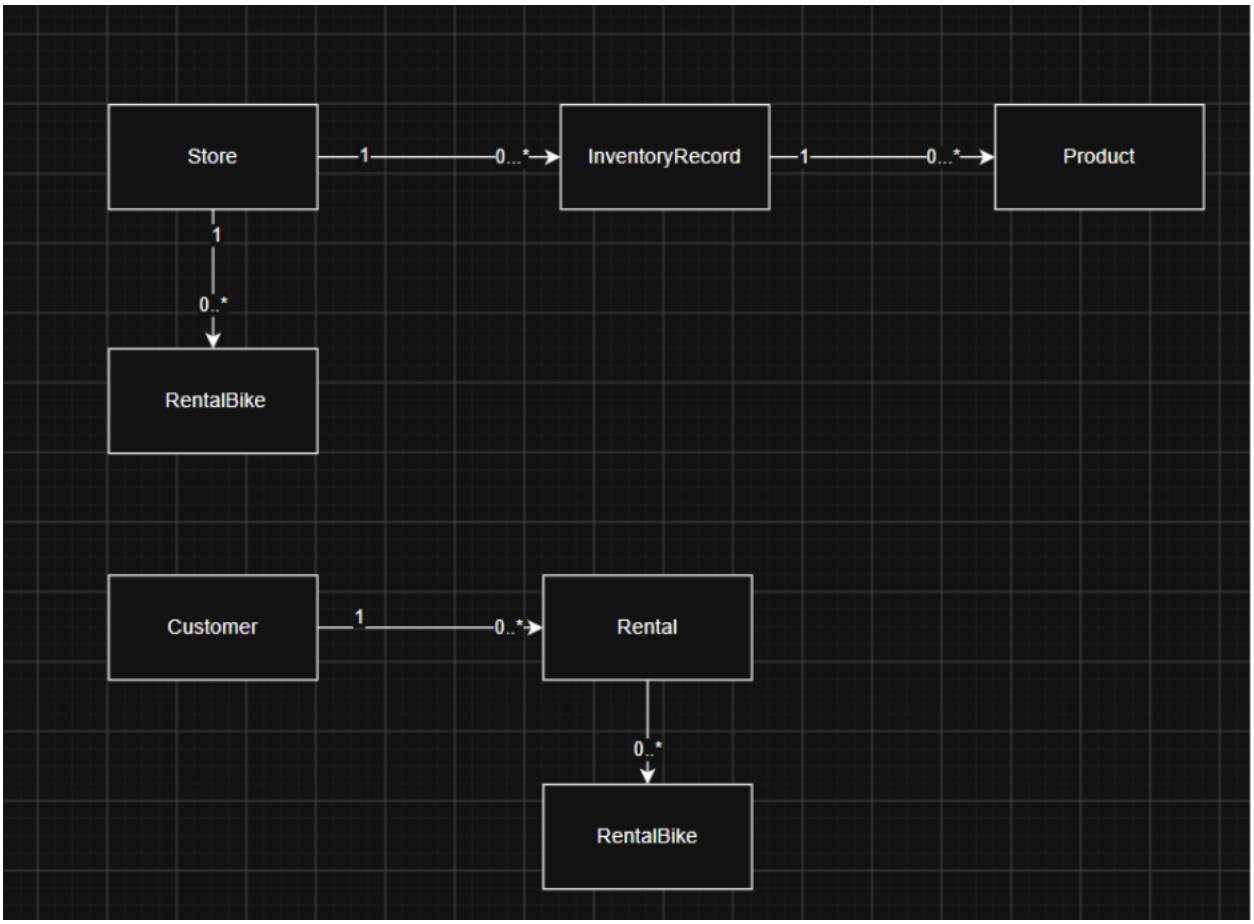
N-tier architecture diagram – represents the separation of concerns: data, logic, and presentation

Object Model

Based on the requirements in week three, the objects that would be prevalent in the project would be as follows:

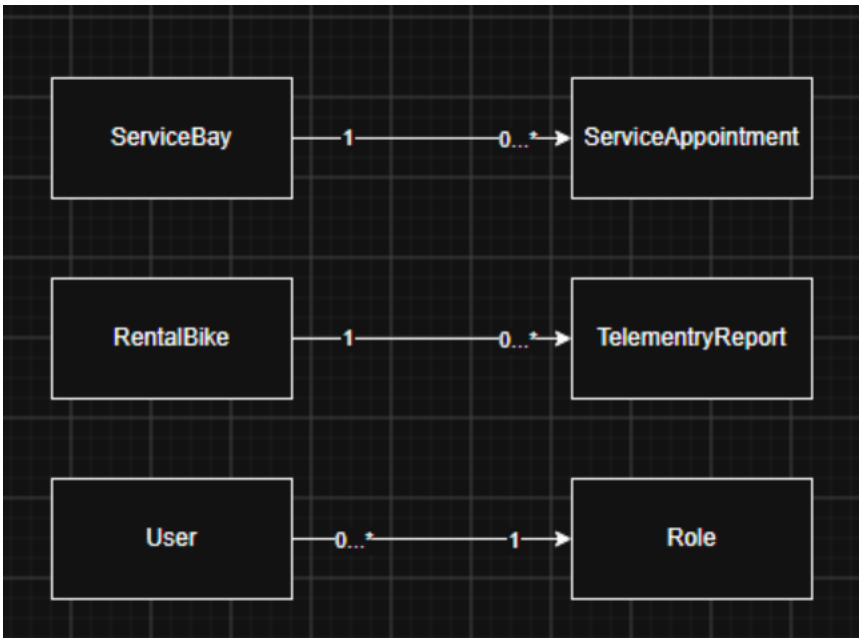
Object	Why it belongs in the model	Important relationships
Store	Inventory is tracked across four locations, and bikes/products have locations.	Store has many Inventory Records; Store may have many Rental Bikes.
Product	Retail employees search products by UPC/name and managers view product inventory and historical sales.	Product has inventory at one or more Stores; Product appears in Sales.
Inventory Record / Stock Level	Inventory count depends on both the Product and the Store, so this works well as the object connecting them.	Belongs to one Product and one Store.
Sale / Sales Transaction	Historical sales must be searchable by date, product, and employee.	Sale involves one or more Products, occurs at a Store, and is associated with an Employee.
User / Account	The system has authenticated employees, managers, and customers with role-based permissions.	User has a Role; a Customer or Employee can be associated with a User account.
Role	Access varies among retail employees, rental employees, managers, service staff, lead service staff, customers, and administrators.	One Role may belong to many Users.
Customer	Customers rent bikes, purchase products, own service bikes, and view their bike's service status.	Customer can have many Rentals and Customer Bikes.
Rental Bike	Rental bikes have type, status, location, and telemetry information.	Rental Bike belongs to a Store/location, can appear in many Rentals over time, and has many Telemetry Reports over time.
Rental	A rental connects a customer to one or more bikes and stores information needed for rental history.	One Rental belongs to one Customer and at least one bike.

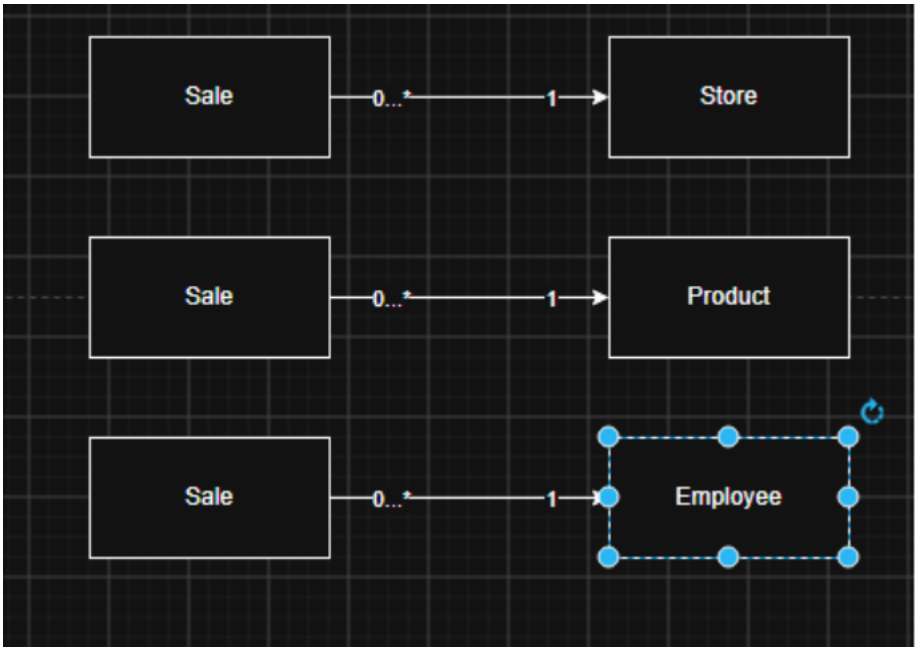
Customer Bike	Customer-owned bikes are serviced separately from rental bikes and are associated with customer accounts.	Customer owns zero or more Customer Bikes; Customer Bike can have Service Appointments.
Service Bay	Service bookings are organized around individual bays and their availability.	Service Bay has many Service Appointments; service staff may be assigned to a Bay.
Service Appointment / Booking	The requirements explicitly call for creating, changing, deleting, and checking scheduling conflicts for bookings.	Appointment uses one Service Bay and involves either a Customer Bike or Rental Bike.
Telemetry Report	E-bikes expose timestamped battery, mileage, and location information.	Telemetry Report belongs to one Rental Bike, specifically an e-bike.



ER Diagram - maps Store, InventoryRecord, Product, RentalBike, Customer, Rental with

cardinalities. Shows how stores own inventory, customers create rentals, and bikes belong to stores. This is the entity-relationship diagram for the retail and rental domains.





ER Diagrams - Extends the data model with ServiceBay, ServiceAppointment, TelemetryReport, User, Role, Sale, Employee, Product, Store. Defines one-to-many relationships for the data-design section, showing persisted entities and relationships.

Services

The Golden Bike Shop System will separate its business logic into services based on the three areas identified in the requirements document: Retail sales, bicycle rentals, and bicycle service. Supporting services will handle authentication/authorization and integrations using external systems. This separation keeps responsibilities clear while still allowing the services to operate as part of the same application.

Service	Responsibility	Communicates With
Inventory Service	Maintains product information and inventory counts for each Golden Bike Shop location. Supports searching products by UPC or name and retrieving inventory across all four stores.	Product and inventory data store, Sales Service, external point-of-sale/inventory system, web interface
Sales and Reporting Service	Provides historical sales information and allows managers to filter sales by date, time, product, and employee. It also supplies inventory-related sales history used by store managers.	Inventory Service, sales data store, manager interface
Rental Service	Manages rental bikes, including their type, current status, availability, and rental transactions. It verifies that a bike is available before allowing it to be rented and stores completed rentals for historical lookup.	Customer data, Rental Bike data, Service Management Service, Rental Reporting functions
Service Scheduling Service	Manages service appointments and service bay availability. It creates, modifies, and deletes appointments and checks for scheduling conflicts before reserving a bay.	Service Bay data, Service Management Service, service staff interface
Service Management Service	Tracks the servicing of both customer-owned bikes and rental bikes. For customer bikes, it maintains statuses such as Awaiting Service, In Service, and Ready for Pickup and associates the bike	Service Scheduling Service, Customer data, Rental Service, service staff and customer interfaces

	with its service appointment and assigned bay.	
Telemetry Integration Service	Retrieves and persists e-bike telemetry information from the existing telemetry source. This includes battery charge, mileage, location, and timestamps.	External telemetry system, Rental Service, rental employee interface
Account and Authorization Service	Manages authenticated users and determines which information and functions each user is permitted to access based on role. Manager-level functions remain restricted to authorized managers and administrators.	All application services and user interfaces
Payment Integration	Connects customers to the existing external payment-processing system when an in-stock online product is purchased. The Golden Bike Shop system does not replace the payment processor itself.	Inventory Service, customer-facing web interface, external payment processor

The Inventory Service supports both retail employees and store managers. Retail employees need to search for products and see inventory levels at all four locations, while managers need a detailed inventory view containing product information, on-hand quantities, and historical sales information. The service will also support online purchases by determining whether a product is currently in stock before the customer is directed to the external payment system.

The Rental Service is responsible for the rental fleet. Rental bikes have a type of either traditional or e-bike and a status of available, rented, or servicing. Before creating a rental, the service checks the selected bike's status and rejects the transaction if the bike is already rented or serviced. It also retains completed rental transactions, so rental managers can search historical rentals by customer, rental item, or date.

The Service Scheduling Service and Service Management Service divide the service department's responsibilities. Scheduling is concerned with the calendar, appointments, service bays, and conflict detection, while service management is concerned with the actual progress of a bicycle through the service process. This allows scheduling changes to remain separate from updates such as marking a customer's bicycle as Awaiting Service, In Service, or Ready for Pickup.

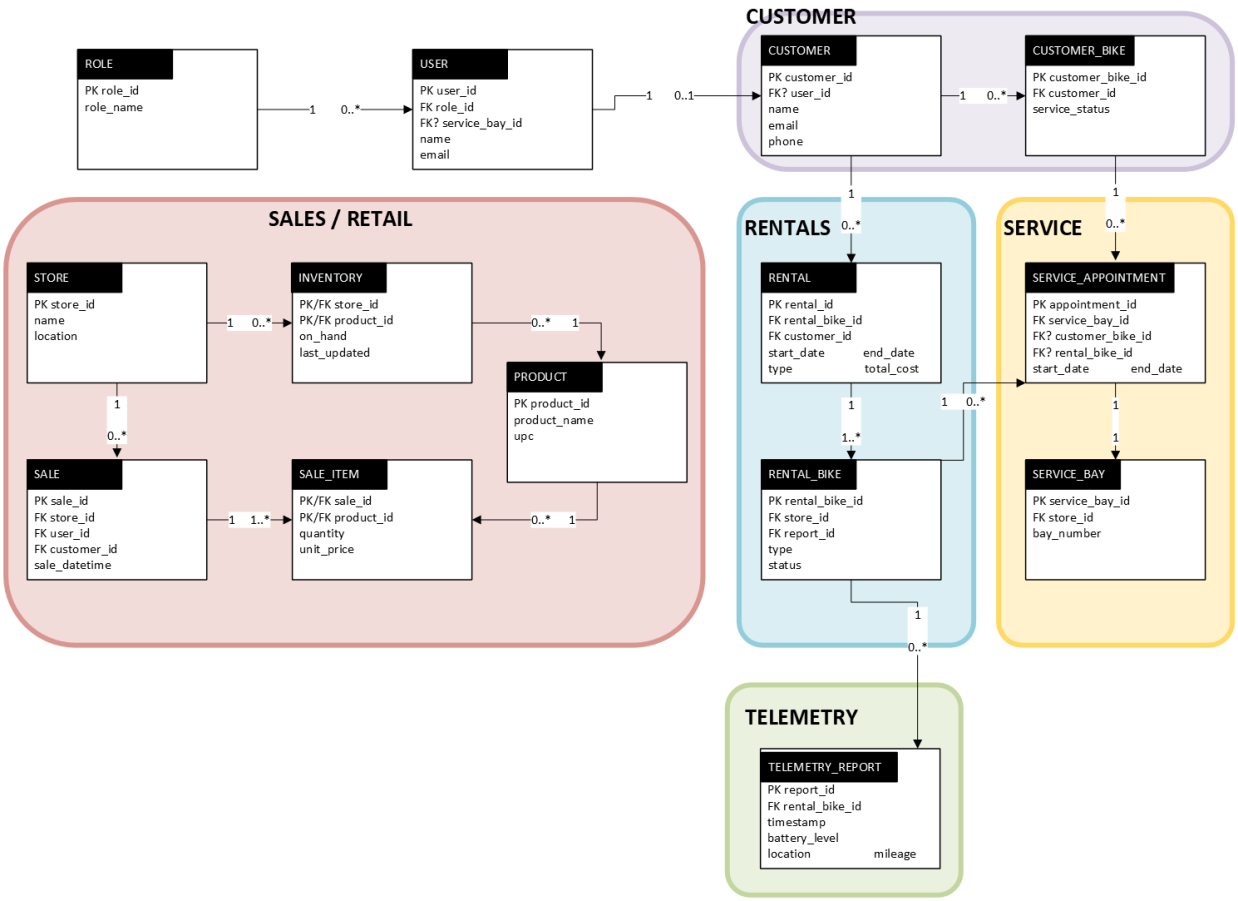
The Telemetry Integration Service provides a boundary between the Golden Bike Shop application and the existing e-bike telemetry system. The requirements state that telemetry is already available from another system, so this service retrieves that information rather than creating a second telemetry platform. It provides battery, location, mileage, and timestamp information to rental staff through the unified application. This data is retained along with its timestamp to preserve the last-known state.

Many of these services allow for scheduled jobs based on recurring leadership or employee operations. It would be useful to have the Telemetry Integration Service pull data at a set interval so that the system could flag unhealthy bikes. The Inventory Service and Sales and Reporting Service would likely be used on a daily basis, or to pull weekly or monthly summaries. These frequent-access reports could be generated through a job to remove the need for manual requesting.

Data Design

The data design for the Golden Bike Shop system defines how the core business entities are stored, related, and accessed within the unified database. This section connects the object model to persistent storage and ensures the architecture is grounded in a clear, consistent schema.

This simplified diagram shows our first pass at the data model, organized by business domain: Sales/Retail, Rentals, Service and Telemetry. User, role and customer records at the top connect to all of them. A more detailed version, including bridge entities, is in Section 05.



Below the table explains what was persisted and why:

Object	Persisted?	Reason
Store	Yes	The system needs to know which of the four locations owns inventory, rental bikes, and service bays.
Product	Yes	Product information such as UPC and name must remain available for inventory lookup and historical sales.
Inventory Record	Yes	Inventory quantities must be stored for each product at each store and remain synchronized with sales data.
Sale	Yes	Historical sales must be available to managers and searchable by date, product, and employee.
Sale Item	Yes	This linking record connects products to individual sales when one sale can contain multiple products.

User	Yes	User accounts and assigned roles must be retained so the system can enforce role-based access.
Role	Yes	Roles such as manager, rental employee, and service staff are needed to determine what functions a user can access.
Customer	Yes	Customers must be associated with rentals and customer-owned bikes, and customers need accounts to view service information.
Rental Bike	Yes	Each bike's type, location, and current status must remain available after the user leaves and returns to the system.
Rental	Yes	Completed and active rental transactions must be stored so managers can retrieve rental history.
Customer Bike	Yes	Customer-owned bikes must be associated with customer accounts and retain their current service status.
Service Bay	Yes	The system must retain the available service bays so appointments can be assigned to them and checked for conflicts.
Service Appointment	Yes	Appointment start time, end time, bay assignment, and associated bike must remain available for calendar scheduling.
Telemetry Report	Yes	Live telemetry is always available from an existing external system. For operational purposes, Golden Bike Shop retains the most recent report.

How the Architecture Meets the Quality Attributes

NFR-001: Changes to operational data shall be committed to the database and visible to all users within 2 seconds of the transaction.

This requirement is fulfilled by the full chain of our architecture. It requires atomic operations performed by the backend application on the database, but also requires the frontend to make immediate calls out to the backend application. The backend application uses its services for inventory, sales, status changes, etc. to make near-instantaneous changes to the corresponding tables.

NFR-002: The system shall operate correctly on the current versions of the organization-approved web browsers used at all four Golden Bike Shop locations without requiring installation of specialized client software.

This requirement is fulfilled entirely by the UI application. The UI needs to be highly compatible, but the requirement keeps this achievable by focusing on specific browsers that the organization has selected and approved for use. By keeping actual business logic out of the UI application, we can ensure a lightweight, flexible application that gives a consistent experience regardless of browser.

NFR-003: The system shall enforce role-based access control so that users can access only the functions and information appropriate to their assigned role, with manager-level functions restricted to authorized managers and administrators.

This requirement is fulfilled by the business logic application and the UI working in tandem. The UI should use the role and permissions for any individual user to determine what they see, what they can interact with, and the operations they can perform on any elements. The business logic application also needs to check credentials within its APIs to confirm that the user associated with a request is authorized to make it.

Up-to-date role and permissions data will be supplied by the database, meaning this requirement does depend on the full chain of our architecture.

NFR-004: The system shall include sufficient technical documentation and code organization to allow a developer familiar with the project to understand and modify individual system components without requiring extensive assistance from the original developer.

This requirement is impacted by but not dependent on any of the architecture choices we made. Documentation for the frontend UI application and the backend business logic application would be kept separate, since they are intended to be maintained separately. The one piece that is notable here is that any change to the business logic application will require a redeployment of that application, since the application is hosted as a monolith. That both simplifies and complicates the deployment, since any developer maintaining the system only needs to know how to perform the one type of redeployment, but also needs to confirm no regression within any part of the system.

The documentation should be supplemented with extensive integration testing to minimize unintended cross-service impacts.

NFR-005: The system shall provide a clear and consistent user interface that allows trained store, rental, and service employees to perform their primary tasks without requiring technical assistance.

This requirement is heavily impacted by our architectural decisions. Because the UI application is a monolith like the backend application, it is susceptible to placing too much information in front of users. NFR-003 should help this somewhat, as users should never be shown parts of the UI they cannot interact with. Still, less tech-savvy users with higher level permissions may struggle at first.

The flip side of having a monolith UI means that the experience within any role will use the same general structure, and consistency is built in.

NFR-006: The system shall be available to authorized users at least 99.5% of the time during normal business operating hours, excluding scheduled maintenance that is communicated to users in advance.

This requirement is fulfilled by the entire architecture but is highly impacted by our decisions. Because the UI contains no business logic, it is fully dependent on the connection to the backend logic application. If it loses that connection, it loses the ability to perform functionality. We made a change to use multiple instances of the business logic application behind a load balancer to enable the system to lose one without going down, but things like a local outage would still take the business fully offline. Having only a single instance of the database containing all data for the business also means that the backend losing connection to the database or the database going down would take the business fully offline.

Our architecture and the other non-functional requirements put this one at risk. They don't come fully at the cost of it, and they even support it during normal operating conditions. During disasters, though, that changes. NFR-002 for example suggests that even the UI application lives externally to the store. During a full outage, store employees would have a cached version of the UI at best. We should have further conversations with the business leadership to determine how they want to balance ongoing costs with stability during disasters.

Case Against the Architecture

We selected N-tier architecture because both the Requirements Document and the UI Design Document already assume a single unified front-end, a single shared application layer, and a single unified database, with role-based access controlling what each user sees.

We asked MS Copilot for the strongest case against that choice, gave it the requirements, constraints, and reasoning, and asked it to attack the decision rather than replace it. It gave

several attacks listed out but the one that stood out the most was that N-tier architecture creates a single, centralized backend that becomes a bottleneck and a single point of failure for a system with multiple independent domains (inventory, rentals, service scheduling, telemetry). By forcing all business logic into one application, N-tier tightly couples unrelated workflows, limits scalability, increases long-term maintenance cost, and makes the system fragile under load. This design choice sacrifices reliability, modifiability, and future growth in exchange for short-term simplicity.

The objection is valid because the Golden Bike Shop system does have multiple domains, telemetry and rentals are more dynamic than inventory or sales, a single backend can become a bottleneck, a monolithic backend can accumulate technical debt, and multi-location operations can suffer if the backend fails. These concerns are real and meaningful. Although the objection is theoretically strong, it is practically weak for this specific system because the business is small (4 stores), traffic volume is low (employees + customers), telemetry is simple (battery + mileage + location).

Only one part of the objection identifies a risk that must be addressed even in a small system - the backend must not become a single point of failure. This will not require changing the architecture, only the deployment strategy to ensure that the backend should run with redundancy (e.g., two instances behind a load balancer), the database should have automated backups and failover, telemetry ingestion should have rate limiting or buffering to avoid overload. We did not change anything in the architecture itself. The N-tier decision still stands, but one operational safeguard will be added to have the backend be deployed with redundancy to avoid total system outage and eliminate the single point of failure.

05 / DATA DESIGN

The relational model governs persistent data. The revised ER diagram and corrected table descriptions include both bike types in service appointments and comprehensive telemetry reports.

How Data Is Stored

The Kickstarters team chose a relational SQL database to store data for the Golden Bike Shop. We chose a relational database because records are consistent in size and format, and tables have strong connections, including bikes, customers, users, rentals, service appointments, products, inventory, and telemetry records. These relationships can be represented clearly through primary and foreign keys, which helps maintain consistency when, for example, a rental connects a customer to a specific bike or a service appointment connects a bike to a service bay. This also fits the current size of the system, which is designed around four store locations and roughly 400 bikes, while still supporting the Week 3 requirements for current inventory data, role-based access, and maintainability. This also supports the Week 3 non-functional requirements for data consistency, reporting accuracy, role-based access, maintainability, and reliable transaction processing.

We considered a document-based NoSQL database as an alternative. NoSQL could provide more flexibility for data whose structure changes frequently, especially telemetry data, but we would give up some of the simplicity of enforcing relationships and referential integrity between the system's core records. Because the Golden Bike Shop depends heavily on connected data across customers, bikes, rentals, inventory, and service activity, a relational model gives us a clearer and more consistent way to query and manage that information.

If telemetry volume grows substantially in the future, the team could revisit a hybrid approach where transactional business data remains relational while high-volume telemetry is stored separately.

Entity Relation Diagram

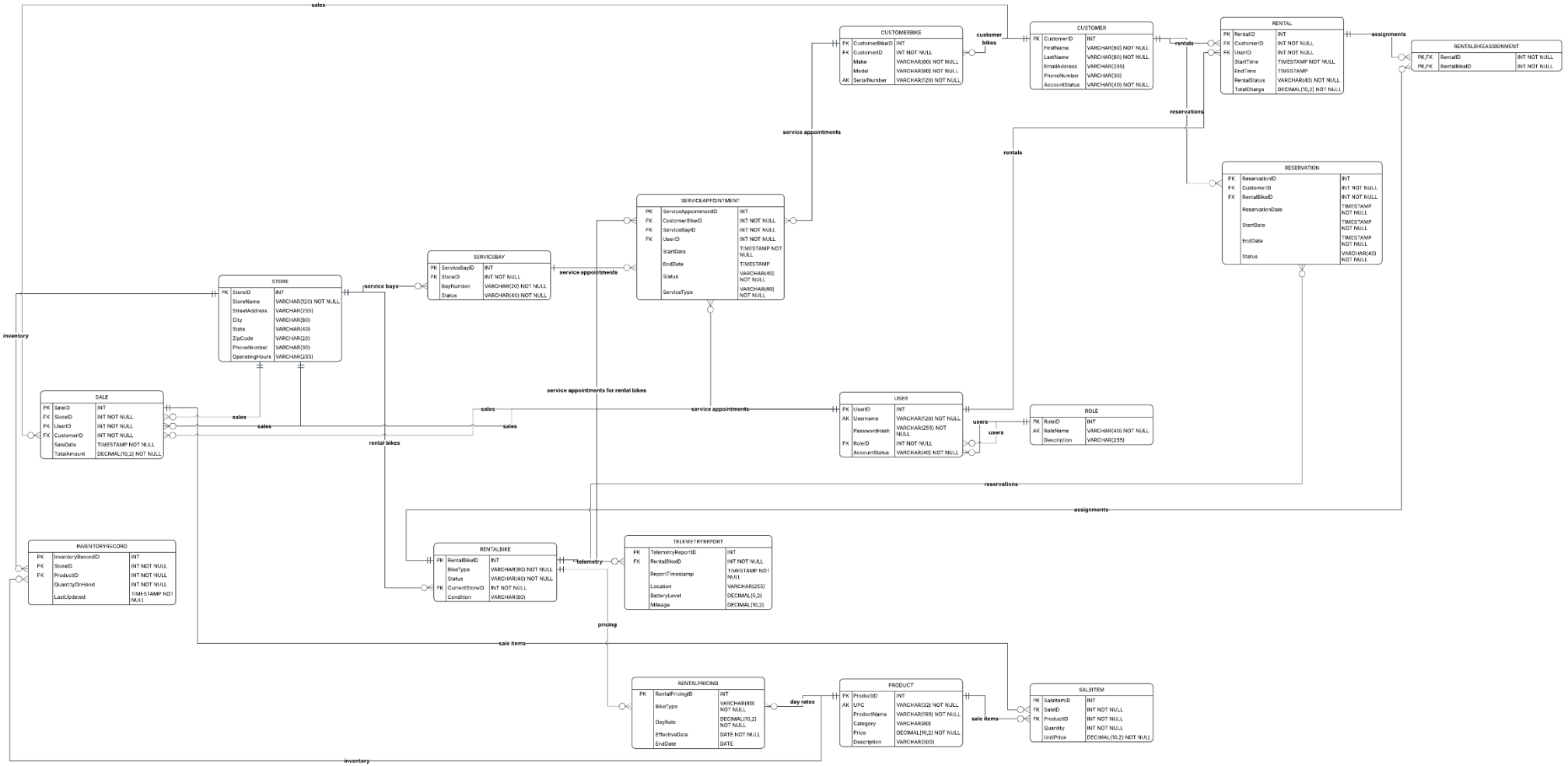


Table Descriptions

Table	Description
Store	Stores information about each Golden Bike Shop location. This can be used to identify where inventory, rentals, employees, and services are located. • **Holds:** StoreID, StoreName, StreetAddress, City, State, ZipCode, PhoneNumber, OperatingHours • **Key:** StoreID • **Related to:** Inventory, RentalBike, ServiceBay, Sale • **Purpose:** Identifies which location owns inventory, rental bikes, and service bays.
Product	Stores information about products sold by the Golden Bike Shop, including the product description, price, category, and other associated metadata. • **Holds:** ProductID, UPC, ProductName, Category, Price, Description • **Key:** ProductID • **Related to:** Inventory, SaleItem • **Purpose:** Represents items sold or stocked across stores.
Inventory	Stores inventory information for each product, including the quantity available and the store location where the product is stocked. • **Holds:** InventoryRecordID, StoreID, ProductID, QuantityOnHand, LastUpdated • **Key:** InventoryRecordID • **Related to:** Store, Product • **Purpose:** Tracks product quantities at each store.
Sale	Stores records of completed sales for transaction history, reporting, and recordkeeping purposes. • **Holds:** SaleID, StoreID, UserID, CustomerID, SaleDate, TotalAmount • **Key:** SaleID • **Related to:** SaleItem, User, Store, Customer • **Purpose:** Stores historical sales for reporting and auditing.
SaleItem	Stores the individual products included in a sale transaction. This entity resolves the many-to-many relationship between sales and products because a sale can contain multiple products and a product can appear in many sales. • **Holds:** SaleItemID, SaleID, ProductID, Quantity, UnitPrice • **Key:** SaleItemID

	• **Related to:** Sale, Product • **Purpose:** Connects products to sales when one sale contains multiple items.
User	Stores login and account information for employees and customers, including the role assigned to each user. • **Holds:** UserID, Username, PasswordHash, RoleID, AccountStatus • **Key:** UserID • **Related to:** Role, Sale, Rental, ServiceAppointment • **Purpose:** Supports authentication and role-based access.
Role	Stores the different roles available within the system, such as customer, employee, manager, or administrator. Roles can be used to determine what actions a user is authorized to perform. • **Holds:** RoleID, RoleName, Description • **Key:** RoleID • **Related to:** User • **Purpose:** Defines permissions for each user type.
Customer	Stores customer information, including contact information and other details needed for purchases, rentals, and service appointments. • **Holds:** CustomerID, FirstName, LastName, EmailAddress, PhoneNumber, AccountStatus • **Key:** CustomerID • **Related to:** Rental, Reservation, CustomerBike • **Purpose:** Represents customers who rent bikes or bring bikes for service.
Rental Bike	Stores information about bicycles available for rental, including the bicycle's type, status, condition, and current store location. • **Holds:** RentalBikeID, BikeType, Status, CurrentStoreID, Condition • **Key:** RentalBikeID • **Related to:** Store, RentalBikeAssignment, Reservation, RentalPricing, ServiceAppointment • **Purpose:** Tracks rental bikes and their availability.
Rental	Stores records of bicycle rentals, including the customer, rental bicycle, rental dates, location, and status of the rental. • **Holds:** RentalID, CustomerID, UserID, StartTime, EndTime, RentalStatus, TotalCharge • **Key:** RentalID • **Related to:** Customer, User, RentalBikeAssignment • **Purpose:** Stores rental transactions and history.

RentalBikeAssignment	Stores the relationship between rentals and rental bikes. This entity allows a single rental transaction to include multiple bicycles and allows a bicycle to participate in many rental transactions over time. • **Holds:** RentalID, RentalBikeID • **Key:** RentalID + RentalBikeID • **Related to:** Rental, RentalBike • **Purpose:** Supports many-to-many rentals involving multiple bikes.
RentalPricing	Stores rental pricing rates for bicycle categories or rental inventory. This allows the system to maintain pricing information separately from bike records and supports future rate changes. Historical pricing records allow active rentals to retain the original rental rate even if rates change before the bike is returned. • **Holds**: RentalPricingID, BikeType, DayRate, EffectiveDate, EndDate • **Key**: RentalPricingID • **Related to**: RentalBike • **Purpose**: Maintains rental rates and supports historical pricing changes.
Reservation	Stores future rental bookings made by customers. A reservation associates a customer with a specific rental bike for a defined time period and allows the system to manage bike availability before the rental begins. • Holds: ReservationID, CustomerID, RentalBikeID, ReservationDate, StartDate, EndDate, Status, • Key: ReservationID • Related to: Customer, RentalBike • Purpose: Stores future bike reservations.
Customer Bike	Stores information about bicycles owned by customers that may be brought to the Golden Bike Shop for maintenance or repair. • **Holds:** CustomerBikeID, CustomerID, Make, Model, SerialNumber • **Key:** CustomerBikeID • **Related to:** ServiceAppointment, Customer • **Purpose:** Represents customer-owned bikes brought in for service.
Service Bay	Stores information about the service bays available at each store location and their availability for bicycle maintenance and repair work. • **Holds:** ServiceBayID, StoreID, BayNumber, Status

	• **Key:** ServiceBayID • **Related to:** Store, ServiceAppointment • **Purpose:** Tracks available service bays for scheduling.
Service Appointment	Stores information about scheduled service appointments, including either a customer bicycle or rental bicycle, service bay, store location, appointment date and time, and current appointment status. • **Holds:** ServiceAppointmentID, CustomerBikeID, RentalBikeID, ServiceBayID, UserID, StartDate, EndDate, Status, ServiceType • **Key:** ServiceAppointmentID • **Related to:** CustomerBike, RentalBike, ServiceBay, User • **Purpose:** Stores service scheduling and history.
TelemetryReport	Stores telemetry information received from connected rental bikes, such as location, battery level, mileage, or usage statistics. This data may be retained for historical reporting and fleet monitoring or displayed in real time from an external telemetry provider. • **Holds:** TelemetryReportID, RentalBikeID, ReportTimestamp, Location, BatteryLevel, Mileage • **Key:** TelemetryReportID • **Related to:** RentalBike • **Purpose:** Stores historical telemetry information received from connected rental bikes.

How Data Moves

Scenario 1: Creating a Reservation

When a customer creates a reservation, the process begins in the **Presentation Tier**, which is the web or mobile web interface used by customers and employees. The customer enters reservation details such as the desired rental dates, store location, and selected bikes. The Presentation Tier is responsible for displaying information, collecting user input, and sending the request to the system for processing. No business rules or database operations occur at this layer.

The request is then passed to the **Middle Tier**, which contains both the **Business Logic Layer** and the **Data Access Layer**. The Business Logic Layer validates the reservation by checking that the customer account is active, the selected dates are valid, and the requested bikes are available during the reservation period. Once all business rules have been satisfied, the Business Logic Layer instructs the Data Access Layer to create the necessary reservation records.

The **Data Access Layer** serves as the connection between the application's business processes and the database. It receives the validated data from the Business Logic Layer and performs the

required database operations. For a reservation, it inserts a record into the Reservation entity containing the customer, bike, reservation dates, and status information.

Finally, the information is stored within the **Data Store Layer**, which consists of the relational database. The reservation data becomes part of the permanent system record and can later be retrieved by customers, employees, and reporting functions throughout the application.

Scenario 2: Starting a Rental

The rental process begins in the Presentation Tier when an employee selects an existing reservation or creates a walk-in rental transaction. The employee verifies customer information and confirms which bikes are being rented. The Presentation Tier submits this information to the Middle Tier for processing.

Within the Middle Tier, the Business Logic Layer validates that the bikes are available, ensures the customer is eligible to rent, and determines the applicable rental rates. Business rules related to pricing, rental duration, and inventory status are applied at this point. Once validated, the Business Logic Layer passes the transaction to the Data Access Layer.

The Data Access Layer creates the Rental and RentalBikeAssignment records and updates the associated RentalBike records. These updates are sent to the Data Store Layer, where the database persists the new rental information and inventory changes.

This approach ensures that all rental transactions follow consistent business rules before any changes are made to the database and prevents inventory inconsistencies across the system.

Scenario 3: Closing a Rental

When a customer returns one or more bikes, the process again begins in the Presentation Tier. An employee enters the return information, confirms the condition of the bikes, and initiates the rental closure process through the user interface.

The request moves to the Middle Tier, where the Business Logic Layer calculates any applicable charges, such as late fees or additional rental costs. It also determines whether any maintenance records need to be generated based on the returned bike condition. Once the calculations and validations are complete, the Business Logic Layer sends the required updates to the Data Access Layer.

The Data Access Layer updates the Rental record with the return date, updates the rental transaction and stores the final charges associated with the rental, updates bike availability, and stores maintenance information when necessary. These transactions are written to the Data Store Layer, ensuring that inventory, financial records, and rental history remain accurate and synchronized.

Changes Made to the Week 5 Design

During the development of the data design, several modifications were made to the Week 5 architecture and data model to support real-world business processes. The most significant change was the addition of the RentalBikeAssignment bridge entity, which allows a rental transaction to contain multiple rental bikes while maintaining historical rental records. The reservation process was intentionally limited to a single bicycle per reservation to simplify scheduling, reduce booking conflicts, and better align with the business requirements identified by the team. Finally, maintenance tracking and bike transfer requirements resulted in additional relationships that were not fully represented in the initial Week 5 design. These enhancements ensure that data can move efficiently through the Presentation Tier, Middle Tier, and Data Store Layer while supporting all required business scenarios.

Checking Generated Model

We generated this list for edge cases:

Bike moved between stores or to/from the service center

Rental bikes can be transferred for inventory balancing or repairs. The system must:

- update the bike's store location
- adjust inventory counts
- prevent reservations at the wrong store
- maintain rental history even after relocation

Price changes while a bike is out

If rental pricing changes mid-rental, the system must:

- preserve the original price for the active rental
- apply new pricing only to future rentals
- ensure historical accuracy for reporting

Overlapping service appointments

A service bay cannot be double-booked. The system must enforce:

- no overlapping time windows
- no scheduling a bay that belongs to a different store

- no scheduling a bike that is currently rented out

Rental not returned on time

Late returns affect availability. The system must:

- update expected availability
- notify staff
- prevent new reservations for bikes still marked “out”

Telemetry outage

If the external telemetry system fails or sends null data:

- the reservation system must continue functioning
- the UI must show “telemetry unavailable” rather than crashing
- stale telemetry must not overwrite valid data

Bike becomes unavailable mid-reservation creation

If another clerk rents the bike at the same moment:

- the system must detect the conflict
- fail gracefully
- allow the clerk to choose a different bike

Service appointment created for a bike currently rented

The system must prevent scheduling service for a bike that is not physically present.

User role changes mid-shift

If a user’s role is updated (e.g., promoted to manager):

- permissions must update immediately
- active sessions must refresh access rights
- audit logs must reflect the correct role at the time of each action

Store temporarily offline

If a store loses internet connectivity:

- local UI must degrade gracefully
- rentals may need offline caching

- data must sync once connectivity returns

Bike retired from inventory

When a bike is permanently removed:

- future reservations must be blocked
- historical rentals must remain intact
- inventory counts must update correctly

Any concurrent operations

- Online and in-person sales for a single item
- Two separate clerks renting out the same bike
- In cases where there may be two service leads at the same time, attempts to schedule the same bay for the same or overlapping times
- A sale price activating at the same time a customer is buying the corresponding item

We then gave Copilot just the overviews from our Project Charter and our Requirements document, and asked it to give us a data model for the rental domain. It returned an extensive model that's included in full in the AI Use Record. The individual entities are included here, along with a summary of their relationships.

- **Bike**
  - bike_id (PK)
  - product_id (FK → Product)
  - serial_number (unique)
  - store_id (FK → Store) — current physical home location
  - is_ebike (boolean)
  - battery_health_score (numeric 0–1 or 0–100)
  - status (FK → BikeStatus)
  - created_at
  - updated_at
- **BikeStatus**

- status_code (PK)
  - Enum values: available, rented, reserved, in_service, offline, unavailable_other
- Description

- **TelemetrySnapshot**
  - bike_id (PK, FK → Bike)
  - latitude
  - longitude
  - battery_percentage
  - last_reported_at — timestamp of telemetry arrival
  - updated_at — timestamp of DB update
- **Reservation**
  - reservation_id (PK)
  - bike_id (FK → Bike)
  - customer_id (FK → Customer)
  - start_time
  - end_time
  - created_at
  - updated_at
  - status (enum: active, cancelled, expired)
- **Rental**
  - rental_id (PK)
  - bike_id (FK → Bike)
  - customer_id (FK → Customer)
  - checkout_time
  - expected_return_time
  - actual_return_time (nullable)
  - origin_store_id (FK → Store)

- return_store_id (FK → Store, nullable)
- created_at
- updated_at
- status (enum: active, completed, late, cancelled)

- **ServiceJob**
- service_job_id (PK)
- bike_id (FK → Bike)
- start_time
- end_time (nullable)
- status (enum: scheduled, in_progress, completed, no_show)
- created_at
- updated_at

- **Store**
- store_id (PK)
- name
- address
- timezone
- created_at
- updated_at

- **Customer**
- customer_id (PK)
- first_name
- last_name
- email
- phone
- created_at
- updated_at

- **Role**
  - role_id (PK)
  - name
    - Enum values: owner, store_manager, rental_manager, rental_staff, service_staff, service_staff_lead, sales_staff
- **User**
  - user_id (PK)
  - name
  - email
  - role_id (FK → Role)
- **RoleDomainPermission**
  - role_id (FK → Role)
  - domain (enum: sales, rental, service)
  - can_write (boolean)

Relationships

- **Bike** 1—N **Rental**
- **Bike** 1—N **Reservation**
- **Bike** 1—1 **TelemetrySnapshot**
- **Bike** 1—N **ServiceJob**
- **Customer** 1—N **Rental**
- **Customer** 1—N **Reservation**
- **Store** 1—N **Bike**
- **Store** 1—N **Rental** (origin/return)

Copilot asked for our edge cases, but we told it simply to use its best judgment. Given that, it provided very strong coverage overall. One of our biggest edge cases for the rental domain was a late return on a bike. While not included in the data model summary above, the AI response had

a footnote about late returns automatically affecting availability. For a telemetry outage, its only comment was another footnote about delayed responses resulting in a status change to “offline”. It had one other footnote covering an edge case related to scheduled future rentals. It would prevent overlapping through a constraint on the SQL table itself. While that would absolutely work, it’s a very fragile solution that would be better if codified in business logic.

There were several named edge cases it missed. It has no knowledge of rental pricing, so it is unable to handle any functionality around that. It also doesn’t cover any concurrency concerns as it seems to think they are fully covered by the model itself. That is true on a data level, but the system will need to be built in a way that handles errors deriving from these interactions gracefully.

Perhaps the biggest concern in its model is the placement of “offline” as a rental bike status. If telemetry data being stale or missing can mark a bike as offline, it would hide the true status of the bike, which raises a lot of questions about how the system would handle a bike with that status.

Seeing the Copilot model missing rental pricing data helped us to see that it was missing in our own model as well, so we went back and added it.

06 / QUALITY & TEST PLAN

Testing is integrated into the evolutionary lifecycle, tracing against active functional and nonfunctional requirements and including a bounded restore acceptance gate.

Introduction

The Kickstarters team has developed this Software Quality and Test Plan to establish how we will verify that the Golden Bike Shop system satisfies its functional and non-functional requirements. The system brings together retail inventory, bicycle rentals, e-bike telemetry, and service scheduling across four store locations and approximately 400 rental bicycles.

Our approach focuses on building quality into the software development process rather than waiting until the application is completed to identify problems. This document describes our quality practices, testing responsibilities, procedures for handling defects discovered by customers, five detailed test cases, supplemental testing considerations, and the risks associated with our testing decisions.

Quality Process

The Kickstarters team will use a combination of software quality assurance practices and testing activities to maintain the quality of the Golden Bike Shop system. As discussed in the lecture, software quality assurance encompasses the practices used to create and maintain quality software, while testing is one component of that larger process.

Our quality assurance practices will include peer code reviews, coding standards, requirements reviews, test planning, defect tracking, and continuous improvement. Every significant code change will be reviewed by another team member before being incorporated into the main application. These reviews will help identify potential problems, ensure that coding standards are followed, and improve the maintainability of the application.

We will also establish a Definition of Done for development tasks. A task will be considered complete when its acceptance criteria have been satisfied, relevant automated tests pass, the code has been reviewed, and any necessary documentation has been updated. Major functionality will also be demonstrated to the appropriate business stakeholder before final acceptance.

Testing activities will include unit testing, API testing, integration testing, acceptance testing, exploratory testing, and non-functional testing. Automated unit and API tests will be used to verify business logic and identify regressions. Integration tests will verify that the front end, backend, database, and external integrations communicate correctly. Acceptance testing will determine whether the system satisfies the requirements established in Week 3.

The distinction is that quality assurance establishes and improves the processes used to prevent defects, while testing evaluates the software's actual behavior. Our team will use both to ensure that quality is continuously maintained throughout development. The distinctions between the two are listed below:

Quality Assurance Activities

- User story reviews
- Database design reviews
- Peer code reviews
- Coding standards and conventions
- Definition of Done
- Acceptance criteria validation
- Development process compliance

Testing Activities

- Unit testing
- Integration testing
- System testing
- Regression testing
- User acceptance testing
- Defect verification testing

Quality assurance focuses on preventing defects from occurring, while testing focuses on detecting defects that may already exist.

Testing Inside the Process

In Week 2, the Kickstarters team selected an evolutionary lifecycle with Agile practices. We chose this approach because the Golden Bike Shop requirements involve several operational areas and existing systems, and stakeholder feedback may reveal requirements that need clarification or refinement.

Testing will occur throughout development rather than being postponed until the end of the project. Each iteration will include implementation, automated testing, peer review, and

stakeholder feedback. This allows problems to be discovered and resolved before additional functionality is developed on top of an incorrect implementation. When a user story is selected for development, test scenarios are identified from the acceptance criteria before implementation begins.

Developers will be responsible for creating unit tests and verifying the functionality they implement. Once a feature is ready, another team member will perform independent testing against the user story and its acceptance criteria. For example, after the rental functionality has been developed, another developer will verify that bicycles cannot be rented while their status is rented or servicing.

After the stories assigned to an iteration have been completed, the team conducts system testing against the original requirements document. User acceptance testing is performed by comparing completed functionality against the user stories and acceptance criteria defined in the requirements document. Any defects identified are returned to the development backlog and corrected before the story is considered complete. Integration testing will be especially important because Golden Bike Shop depends on information shared between retail inventory, rentals, telemetry, and service scheduling. The team will verify that changes made in one operational area are correctly reflected wherever that information is used.

Business stakeholders, including store managers and operational staff, will participate in acceptance testing as functionality becomes available. The owner, Gregg Bishop, will provide final business approval.

Testing responsibilities are divided as follows:

Developer: Unit testing and defect correction

Peer Reviewer: Code review and test review

Team: Integration testing, system testing, and regression testing

Business Stakeholder/Product Owner: User acceptance testing

One challenge with our evolutionary approach is that requirements may change after tests have already been written. We will address this by reviewing the affected acceptance criteria, updating the corresponding tests, and rerunning relevant regression tests whenever a requirement changes. Agile development requires comprehensive regression testing that becomes more difficult because features are delivered incrementally. Unlike a traditional waterfall lifecycle where testing largely occurs after development is complete, Agile requires previously completed functionality to be retested whenever changes are introduced. This increases testing effort but reduces the risk that defects accumulate until the end of the project.

Escaped Defects

Despite our testing efforts, some problems may not be discovered until the Golden Bike Shop system is being used by employees or customers. These are escaped defects, and the team needs a repeatable process for identifying, prioritizing, correcting, and preventing them.

When a problem is reported, the development team will create a defect record describing the issue, the affected functionality, the steps needed to reproduce it, and its impact on business operations. Reports may come from employees, managers, customers, application logs, or automated monitoring.

The team will first determine whether the issue is a technical bug or a mismatch between the implemented functionality and business requirements. As discussed in the lecture, a coding or system problem may require a technical correction, while a problem caused by incomplete or misunderstood requirements may require additional discussion with the business stakeholders.

The issue is recorded with detailed reproduction steps, affected functionality, business impact, screenshots when available, and severity. Issues will be prioritized according to their impact. Problems involving unauthorized access, incorrect rental availability, or major interruptions to store operations will receive the highest priority. Less severe problems, such as minor interface inconsistencies, may be scheduled for a later iteration.

Critical issues will be escalated immediately to the development lead and the relevant operational manager. Gregg Bishop will be informed when a problem significantly affects business operations, creates a substantial security concern, or requires a decision involving project scope or cost. Problems involving external integrations may also require coordination with the responsible third-party provider.

Severity classifications are defined as follows:

Critical: Prevents rentals, inventory lookup, service scheduling, or system access.
High: Significant business impact with limited workaround.
Medium: Functionality works but causes inconvenience or reduced usability.
Low: Cosmetic or minor display issue.

The escalation path is:

1. Customer or employee reports issue.
2. Store manager or support contact logs defect.
3. Product owner or project lead determines priority.
4. Lead developer assigns the defect.

5. Developer implements a fix.
6. Tester reproduces the original defect.
7. Regression testing is performed.
8. Product owner verifies resolution.
9. Defect is closed.

Critical defects affecting bike availability, inventory accuracy, service scheduling, or role-based access control are escalated immediately to the project lead and lead developer. These issues may require an emergency hotfix outside the normal sprint cycle.

A defect is considered verified only when the original failure can no longer be reproduced and regression testing confirms that no new defects were introduced.

When necessary, the team will temporarily disable affected functionality, remove inaccurate information from customer-facing views, or direct employees to the existing operational system until the problem can be corrected.

After identifying the cause, a developer will implement the fix and create or update automated tests to prevent the same problem from occurring again. Another team member will verify the correction in a controlled environment and perform regression testing on related functionality before the fix is released.

The team will also conduct root cause analysis on significant or recurring problems. For example, if an unexpected inventory record causes the application to fail, we may introduce automated validation or monitoring that detects similar records before they affect users.

Detailed Tests

The following five tests are based on the Week 3 requirements and their acceptance criteria. Each test will be performed in an isolated test environment using controlled data. Test records must be reset to their stated initial conditions before execution.

Requirements Traceability Overview:

The REQ and NFR identifiers associated with each test case provide traceability between the Week 3 Requirements Document and the verification activities described in this plan. This mapping ensures that both functional and non-functional requirements can be validated through testing.

Test 1: Product Search and Inventory Availability (REQ-001)

Test Type: Integration Testing and Acceptance Testing

User Story: As a retail employee, I want to look up a product by UPC or name and see how much is available at my store and the other three locations.

Acceptance Criteria: Searching by UPC displays product information. Searching by product name displays product information. Product information includes inventory counts for all four store locations.

Requirements Traceability:
This test verifies REQ-001 by validating that retail employees can search for products using either a UPC or product name and view accurate inventory quantities across all four store locations.

Preconditions and Test Data: An authorized retail employee is logged into the test environment. A test product named Test Cable Lock, with UPC 012345678905, exists in the database. Its inventory counts are configured as Store 1: 4, Store 2: 0, Store 3: 2, and Store 4: 7. No inventory transactions occur during testing.

Test Steps:

Step 1: Navigate to the retail inventory search screen.

Step 2: Enter UPC 012345678905 and select Search.

Step 3: Record the product information and inventory counts displayed.

Step 4: Clear the search and enter Test Cable Lock.

Step 5: Select Search and record the resulting information

Expected Result: Both searches return the same product. The application displays the correct UPC, product name, and inventory counts of 4, 0, 2, and 7 for the four respective stores.

Failure Criteria: The test fails if either search does not return the product, returns an incorrect product, omits a store, or displays an inventory count that does not match the configured test data.

Test 2: Rental Bike Availability Restrictions (REQ-004)

Test Type: Acceptance Testing and Integration Testing

User Story: As a rental employee, I want to rent bikes out to customers only when those bikes are available.

Acceptance Criteria: An authorized rental employee can rent a bike with an available status. Rentals are denied for bicycles with rented or servicing statuses.

Requirements Traceability:
This test verifies REQ-004 by confirming that bicycles can only be rented when their status is Available and that rental attempts are rejected when bicycles are already Rented or in Servicing status.

Preconditions and Test Data: An authorized rental employee is logged in. Test customer C-001 exists. Three rental bicycles are configured: RB-101 is Available, RB-102 is Rented, and RB-103 is Servicing. These bicycles have no conflicting test transactions beyond their configured statuses.

Test Steps:

Step 1: Locate bicycle RB-101 and attempt to rent it to customer C-001.

Step 2: Verify that the rental is successfully created.

Step 3: Locate bicycle RB-102 and attempt to rent it to the same customer.

Step 4: Locate bicycle RB-103 and attempt to rent it to the same customer.

Step 5: Return to the rental records and confirm which transactions were created.

Expected Result: The system successfully creates a rental for RB-101 and records that bicycle as rented. Attempts involving RB-102 and RB-103 are denied, and no additional rental transactions are created for those bicycles.

Failure Criteria: The test fails if an available bicycle cannot be rented, an unavailable bicycle can be rented, the successful transaction is not saved, or either rejected attempt creates a rental record.

Test 3: Service Appointment Scheduling Conflicts (REQ-006)

Test Type: Integration Testing and Acceptance Testing

User Story: As a lead service staff member, I want to manage service bookings so that I can maximize bay usage uptime.

Acceptance Criteria: Authorized lead service staff can create appointments with start times, end times, and bay numbers. Conflicting appointments are rejected with an error message, while non-overlapping appointments are accepted and displayed correctly on the calendar.

Requirements Traceability:
This test verifies REQ-006 by validating that service appointments can be scheduled for

available service bays while preventing overlapping appointments that would create scheduling conflicts.

Preconditions and Test Data: An authorized lead service staff member is logged in. Service Bay 2 exists and has an appointment scheduled on November 12, 2026, from 10:00 AM to 11:00 AM. No other appointments exist for Bay 2 that day.

Test Steps:

Step 1: Open the service appointment calendar for Bay 2.

Step 2: Attempt to create an appointment from 10:30 AM to 11:30 AM.

Step 3: Verify that the system rejects the appointment and displays a scheduling conflict message.

Step 4: Attempt to create another appointment from 11:00 AM to 12:00 PM.

Step 5: Save the appointment and refresh the calendar.

Expected Result: The overlapping appointment is rejected without being saved. The appointment beginning at 11:00 AM is accepted because it does not overlap with the existing appointment. The calendar displays both valid bookings with their correct times.

Failure Criteria: The test fails if overlapping appointments are accepted, non-overlapping appointments are rejected, required error messages are missing, or the calendar displays incorrect booking information.

Test 4: Inventory Synchronization Time (NFR-001)

Test Type: Non-Functional Performance Testing

Requirement: Changes to operational data shall be committed to the database and visible to all users within 2 seconds of the transaction.

Acceptance Criteria: A purchase reduces the on-hand inventory count for the store where it occurs. The inventory update must be reflected within two seconds.

Requirements Traceability:
This test verifies NFR-001 by measuring the elapsed time between a completed inventory transaction and the corresponding update within the Golden Bike Shop application to ensure inventory data remains synchronized within the required two-second threshold.

Preconditions and Test Data: The test environment is connected to a functioning test instance or simulator of the existing point-of-sale integration. A test product has an initial quantity of 10 at Store 1. The source system and application use synchronized clocks.

Test Steps:

Step 1: Verify that the Golden Bike Shop application displays an inventory quantity of 10.

Step 2: Record the time and complete a purchase of one unit through the configured point-of-sale integration.

Step 3: Confirm that the source system recorded the completed purchase.

Step 4: Refresh the Golden Bike Shop inventory display at two seconds after the recorded time.

Expected Result: The inventory count decreases from 10 to 9 within two seconds of the completed purchase.

Failure Criteria: The test fails if the inventory count is incorrect or hasn't updated upon refresh. If the required external integration is unavailable, the test is blocked and cannot be marked as passed.

Test 5: Role-Based Access Restrictions (NFR-003)

Test Type: Non-Functional Security Testing

Requirement: The system shall enforce role-based access control so users can access only the information and functions appropriate to their assigned roles.

Acceptance Criteria: Authorized managers can access manager-level functionality. Regular employees cannot access manager-level functionality. Unauthorized attempts to access restricted functions are denied.

Requirements Traceability:
This test verifies NFR-003 by confirming that users can access only the functionality appropriate to their assigned roles and that unauthorized access attempts are denied through both the user interface and backend services.

Preconditions and Test Data: Two test accounts exist. Account M-001 has the Store Manager role, and account E-001 has the Retail Employee role. Both accounts have valid login credentials. The manager inventory view contains existing test inventory data.

Test Steps:

Step 1: Log in using account M-001.

Step 2: Navigate to the manager inventory view and confirm that the page is accessible.

Step 3: Record the page URL and the corresponding manager-only API request from the browser's developer tools.

Step 4: Log out and log in using account E-001.

Step 5: Attempt to access the same manager inventory page directly using the recorded URL.

Step 6: Submit the previously identified manager-only API request using the retail employee's authenticated session.

Expected Result: Account M-001 can access the manager inventory functionality. Account E-001 is denied access through both the user interface and the backend API. No restricted manager data is returned to the employee. The team will use HTTP 403 Forbidden as the expected response for an authenticated user who lacks permission, subject to confirmation of the final API contract.

Failure Criteria: The test fails if the employee can view restricted inventory-management information, successfully perform a manager-only action, or retrieve protected data through the backend API.

Testing limitations

The Kickstarters team recognizes that testing every possible scenario would be neither practical nor realistic within the project's eight-week schedule, four-person development team, and proposed $65,000 budget. Our testing strategy will prioritize functionality based on business importance, operational risk, and the consequences of failure.

The highest testing priorities will be inventory accuracy, rental availability, service scheduling conflicts, role-based access control, and the consistency of information between integrated systems. These areas directly affect daily operations and have well-defined acceptance criteria.

Additional Testing Considerations:
The team identified several supplemental testing areas that may be incorporated into future testing efforts as project schedule and project scope permit:

- Concurrency testing for rental and reservation processing
- Telemetry outage and degraded-data testing
- Cross-domain integration testing
- Expanded role-based security testing

These additional testing considerations were identified during the team's review of testing gaps and are discussed further in the "What The Plan Misses" section.

We will not attempt exhaustive testing of every possible combination of inputs, user actions, and system states. Instead, we will prioritize representative scenarios, boundary conditions, common

workflows, and known high-risk situations. The accepted risk is that an unusual combination of circumstances may expose a defect not encountered during testing.

We will also limit testing of functionality outside the project's agreed scope. This includes the internal operation of existing payment processors, the physical operation of e-bike telemetry hardware, predictive AI forecasting, and functionality associated with expansion beyond the existing four locations. However, we will still test our own application's interactions with external systems, including payment handoffs where applicable and the presentation of telemetry data.

We will not conduct exhaustive stress testing against hypothetical operations substantially larger than the current four-store, approximately 400-bike business. The accepted risk is that the application may require additional performance improvements if the company expands significantly. We would revisit this decision if expansion becomes a formal requirement or performance monitoring identifies capacity limitations.

These boundaries do not remove the need to satisfy the existing non-functional requirements. Before deployment, the team will verify the inventory synchronization limit, role-based restrictions, functionality in all organization-approved browsers, maintainability expectations, and the requirement that at least 80% of usability test participants complete their assigned tasks successfully.

Testing will be considered sufficient for release when the agreed upon acceptance criteria have been verified, required non-functional tests have passed, critical defects have been resolved, remaining issues have been documented and accepted by the appropriate stakeholders, and the owner has approved the system for deployment.

Among these priorities, bike availability tracking represents the highest business risk to Golden Bike Shop operations. If the system incorrectly displays a bike as available when it has already been reserved or rented, customers could arrive expecting a bike that is unavailable. This could result in canceled reservations, lost sales, and reduced customer trust. For that reason, bike availability, reservation validation, and rental processing receive the highest testing priority. The system design reduces this risk because each bike is stored as a unique record, every reservation is associated with a specific bike, and the business logic validates availability before a rental or reservation is approved.

The team has chosen to perform only limited testing of administrative account setup and role assignment. Employee and manager accounts are created infrequently and involve relatively simple business logic compared to rental processing, inventory tracking, and service scheduling. The likelihood of failure is low, and any failure would primarily affect internal employees rather than customers. The risk accepted is that an employee could temporarily receive incorrect permissions or be unable to access the system until an administrator corrects the issue.

We would revisit this decision if account creation becomes more frequent, if users regularly experience login or role-assignment issues, or if account-management defects begin affecting normal business operations.

The team has also chosen to perform only limited testing of employee profile maintenance functions such as updating contact information. These activities occur infrequently and involve simple data updates rather than complex business transactions. The likelihood of failure is low, and the impact would generally be limited to employees rather than customers. The risk accepted is that contact information may temporarily become inaccurate.

We would revisit this decision if profile updates become more common, if users begin reporting frequent data accuracy issues, or if incorrect employee information begins affecting customer service or business operations.

What The Plan Misses

A.I. Copilot was given the sections above then asked “Describe to me what the plan misses. Provide only gaps and not a full plan.”

It returned the following:

Gap 1: Concurrency Testing

The plan does not explicitly test situations where two employees attempt to rent or reserve the same bike at nearly the same time.

Why this matters:

- Week 6 identified concurrent rentals as an edge case.
- Bike availability was identified by the team as the highest business risk because incorrect availability can cause double-booking, lost sales, and customer dissatisfaction.

Team Verdict:

- Real gap.

What changed:

- Added concurrency testing considerations for rental and reservation processing.

Gap 2: Telemetry Failure Scenarios

The plan does not include tests verifying application behavior when the telemetry system is unavailable, delayed, or returns invalid data.

Why this matters:

- The requirements include e-bike telemetry information.
- The architecture relies on an external telemetry source.
- Week 6 specifically identified telemetry outages as an edge case.

Team Verdict:

- Real gap.

What changed:

- Added telemetry outage and degraded-data scenarios to testing considerations.

Gap 3: Negative Security Testing

The current tests validate successful access and denied access for one role-based access scenario, but do not thoroughly test attempts to access restricted functions across multiple roles.

Why this matters:

- NFR-003 requires role-based authorization.
- The system supports multiple user roles with differing permissions.

Team Verdict:

- Partially real gap.

What changed:

- Access-control testing to include role-based negative cases was considered and added to testing considerations.

Gap 4: Cross-Domain Workflow Testing

The plan tests individual requirements but includes little verification that changes in one domain affect other domains correctly.

Examples:

- Renting a bike affects availability.
- Servicing a bike affects rental eligibility.
- Inventory updates affect customer-facing views.

Why this matters:

- The architecture intentionally integrates retail, rental, service, and telemetry domains into one system.

Team Verdict:

- Real gap.

What changed:

- Added additional integration and workflow validation scenarios to the testing considerations list.

Gaps Raised But Rejected

Enterprise-Scale Load Testing

The AI suggested large-scale performance and stress testing.

Why rejected:

- Golden Bike Shop consists of four store locations and approximately 400 rental bikes.
- The cost of enterprise-scale testing is not justified by the expected system size.

Team Verdict:

- Not a meaningful gap for this project.

Disaster Recovery Testing

The AI suggested extensive failover and disaster recovery testing.

Why rejected:

- Important in large production environments.
- Outside the scope of this academic project.
- Does not directly support the highest-priority functional requirements identified by the team.

Team Verdict:

- Not a meaningful gap for the current project.

Although this test plan provides coverage for the system's highest-priority functions, the team recognizes that no testing effort can completely eliminate risk. During review, several potential gaps were identified.

The plan does not include dedicated concurrency testing to verify behavior when multiple employees attempt to reserve or rent the same bicycle simultaneously. Because bike availability was identified as one of the most important business risks, concurrency issues could potentially create duplicate reservations or conflicting rental transactions. Additional testing considerations were added to address this risk.

The plan also provides limited coverage for telemetry failure scenarios. While telemetry functionality is included within the project scope, the current tests focus primarily on expected system behavior rather than failures within the external telemetry source. Additional testing considerations were added to evaluate system behavior when telemetry data is unavailable, delayed, or invalid.

The security tests verify role-based access restrictions, but they do not exhaustively evaluate every possible unauthorized access scenario. Additional negative testing may be required if security-related issues are discovered during development or acceptance testing.

Finally, the plan focuses primarily on validating individual requirements and business processes. Integrated workflows that span multiple application domains may contain defects that are not visible when features are tested independently. Additional integration validation will be performed where changes in rentals, servicing, inventory management, and telemetry affect one another.

After review, the team determined that enterprise-scale load testing and extensive disaster recovery testing were not appropriate additions for this project due to the size, scope, and academic nature of the system. The team accepts the residual risk associated with those areas and will instead focus testing effort on business-critical functionality such as inventory accuracy, rental processing, service scheduling, system integration, and role-based security.

07 / DEPLOYMENT AND MAINTENANCE OUTLOOK

How the system moves through development, QA and production, how each release can be rolled back safely, and which parts of the design will take the most care to maintain over time.

Each of the three tiers (the browser UI, the backend application and the database) is versioned and can be released on its own. A UI-only change, such as a label fix or a new filter, is released without touching the backend. A database-only change, such as adding an index to speed up inventory searches, is released as a migration script without redeploying the application. The backend is the exception: because its services (inventory, rental, scheduling and the rest) are built and deployed as one unit, any backend change redeploys the whole backend.

Each release is recorded as a baseline that lists the exact version of each tier running together, along with the migration scripts, configuration and release notes. That way we always know which combination is in production and which one to return to. Changes enter through change control: each has a defect or change record. The project lead approves it, and Gregg Bishop approves anything that affects scope, cost or store operations. The amount of testing matches the change:

- a UI-only change is tested against the screens and roles it affects
- a database change is checked for performance and compatibility
- a backend change runs the full regression suite, because one change there can reach every domain

The same build moves through three environments without being rebuilt:

Development → QA/Staging → Production. Promotion requires passing gates:

- peer review and unit tests to reach QA
- integration and regression tests, stakeholder UAT and the owner's approval to reach Production.

Releases happen after store hours (after 7 p.m.), and staff see a maintenance notice at log-in (NFR-006). Because the backend runs as two instances behind a load balancer, we update

instance A first and run smoke tests on the critical paths: inventory lookup, renting a bike and booking a bay. Only then do we update instance B, so the stores never go offline. The first go-live is a pilot at one store before rolling out to all four.

Rollback plan:

- **Triggers:** a failed smoke test, any critical defect as defined in our test plan (it blocks rentals, inventory lookup, scheduling or sign-in), or sustained alerts for responses slower than 3 seconds.
- **Application:** we keep the previous version of each tier ready. For a UI or backend release, the load balancer drains the new instance and we redeploy the prior version of only the tier that changed.
- **Database:** migrations only add to the schema (backward-compatible), so the previous version still runs against it. We take a backup just before each release and restore it only as a last resort, because a restore loses transactions made since the backup.
- **Business continuity:** if needed, we turn off the affected feature and staff use the existing point-of-sale, rental and workshop systems, which stay in place.
- **Afterward:** we run a root-cause analysis and add a regression test so the same failure can't return.

What Will Be Hardest to Maintain

The hardest part to maintain will be the coupling we accepted by choosing one backend and one shared database. The three domains overlap on purpose. Rental status affects service scheduling, inventory feeds both the stores and the online shop, and nearly every table links to Store, Customer or User.

That makes every change, even a small perfective one, need careful impact analysis. A change to how bike status works could break rentals, service bookings and reporting at once. Because the backend deploys as one unit, every backend release also requires a full regression test , not just a test of the changed feature. Over time this risks the technical debt our architecture review warned about, where the code becomes harder to understand and change safely.

A second, related challenge is adaptive maintenance of our integrations. The point-of-sale, payment, telemetry, and workshop systems belong to other vendors. When their interfaces change, our system must change too, on their schedule, and those integrations are the hardest parts to test.

We limit both risks by:

- keeping each domain as a separate, documented module inside the monolith
- meeting NFR-004 so any developer can locate and change a component
- keeping all database migrations under version control
- making automated regression tests part of our Definition of Done

08 / Discrepancy and Resolution Notice

In compiling this comprehensive document, there were decisions from earlier components that needed to be updated based on information that was acquired in later weeks. These differences are listed below.

Requirement REQ-014 created (System Requirements)

Future reservations were discussed by the team and added as a feature during data design. We needed a matching requirement to capture the exact description and acceptance criteria we were committing to for the feature.

Screen requirement consistency (UI Design)

Requirement REQ-010 was listed under a screen it did not apply to and REQ-014 didn't exist yet, meaning their usage in the original document was incorrect. We resolved these.

Telemetry persistence (Application Architecture)

Our Software Architecture document only mentioned retrieval of telemetry data from the external system. During data design, we determined that persisting the most recent telemetry for a given e-bike was important for the resiliency of the system. The Telemetry Integration Service was updated accordingly.

Telemetry reporting data (Data Design)

In our earlier documents, we had extensive telemetry data available, including detailed battery insights and confirmation of unchanged locked settings. During data design, we decided to simplify the data we used and retained. All references to telemetry data from before this decision have been changed to reflect the final design.

Service appointments on rental bikes (Data Design)

Most of our documents correctly tied service appointments to both customer bikes and rental bikes. During data design, we only set our ServiceAppointment relational object to link with customer bikes. This has been fixed to support both customer and rental bikes.

Multi-bike rentals (Application Architecture)

One of the more complicated decisions we made during data design was supporting only a single bike per reservation, but one or more bikes associated with a single rental ID. This required a change to align the concept of a rental in the Software Architecture document with the final design.

Customers as stakeholders (Project Charter)

Our initial charter listed customers as "indirectly affected". This failed to capture the breadth of ways they interact with the business as well as the added functionality around bike service status visibility. Updated the customer stakeholder to clarify this.

Payment interactions (Project Charter)

Our original charter was worded very strongly in a way that suggested the system would not interact with any payment steps, internally or externally. We clarified that customers can start a payment from our system, but it is processed by the payment systems the company already uses.

NFR-001 evolution (System Requirements and all later components)

Our first non-functional requirement has gone through the most changes of any piece of our design. It was first proposed as an asynchronous five-minute SLA for inventory changes in the system, though that was quickly changed to one minute. Once we fully understood our architecture and determined it would be responsive and more synchronous, it was further reduced to 2 seconds. When pulling together this document, we realized we had applied it outside of just inventory updates while designing our screens, and that it was important for things like rental bike status to also be covered by such a requirement. The final version covers operational data as a whole, and changes have been made to several components to reflect it.